DAYANANDA SAGAR COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to VTU, Approved by AICTE
Accredited by National Assessment & Accreditation Council (NAAC) with 'A' grade
Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore - 560 078.

RECORD BOOK

NAME OF THE STUDENT .....

BRANCH .....

U S N ..... SEMESTER .....

SECTION ..... CLASS ROLL No.....

SUBJECT .....

Department of Electronics & Communication Engineering

Advanced Embedded System Laboratory

VII Semester (22EC72)



Staff in-charge:

Dr Manasa R

Prof. Bindu H M

Lab Staffs:

Dr Abhishek M B

Prof. Vibha T G

Prof Sowmya P

Lab Instructors: Mr. Suresh Babu & Mr Shivaswamy

HOD: Dr. Shobha K R

Name of the Student	:	
Semester /Section	:	
USN	:	
Batch	:	

Dayananda Sagar College of Engineering

Shavige Malleshwara Hills, Kumaraswamy Layout, Banashankari,
Bangalore-560111, Karnataka

Tel : +91 80 26662226 26661104 Extn : 2731 Fax : +90 80 2666 0789

Web - http://www.dayanandasagar.edu Email : hod-ece@dayanandasagar.edu

(An Autonomous Institute Affiliated to VTU, Approved by AICTE & ISO 9001:2008 Certified)

2026-2027

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute Affiliated to VTU, Approved by AICTE)
Accredited by National Assessment & Accreditation Council (NAAC) with 'A. grade'
Shavige Malleshwara Hills, Kumaraswamy Layout, Bangalore – 560.079.

Department
of
Electronics and Communication Engineering
Integrated Advanced Embedded systems lab
Code: 22EC72

Laboratory Certificate

This is to certify that Mr./Ms. ..... *bearing USN* ..... *has satisfactorily completed the experiments in above practical subject prescribed by the* ..... Visvesvaraya Technological University *for the* **'VII'** *Semester B.E. in the aforesaid* **Integrated Advanced Embedded systems** *laboratory during the year* **2026-2027**

Date : _____

Marks	
Maximum	**Obtained**

_____
Name of the Faculty

_____
Signature of Faculty Incharge

_____
Head of the Department

About the College

The Dayananda Sagar College of Engineering was established in 1979 by Sri R. Dayananda Sagar. Dayananda Sagar College of Engineering operates under the aegis of the **Mahatma Gandhi Vidya Peetha Trust** is approved by All India Council for Technical Education (AICTE), Govt. of India and affiliated to Visvesvaraya Technological University. It has the widest choice of engineering branches having 18 Under Graduate courses & 6 Post Graduate courses. In addition, it has 20 Research Centres in different branches of Engineering catering to research scholars for obtaining Ph.D under VTU. Various courses are accredited by NBA.

The Institute is spread over 23 acres of land with large infrastructure supported by laboratories with state-of-the-art, Equipment & Machines. The Central Library with modern facilities and the Digital Library provides the knowledge base for the students.

The campus is WI-FI equipped with large bandwidth internet facility. The College has good faculty strength with highest professional integrity and committed to the academics with transparency in their actions. Each faculty takes the responsibility of mentoring a certain number of students' through personal attention paving the way for the students' professional growth. The faculty are research oriented having number of sponsored R & D projects from different agencies such as Department of Science & Technology, Defense R & D organizations, Indian Space Research Organization, AICTE etc..

About the Department

Department of Electronics and Communication Engineering is one of the oldest and biggest department in the Dayananda Sagar College of Engineering with 52 faculty members and 1200+ students. Most of our faculty have pursued Ph.D. and others are in the process. The ECE department provides high-quality, innovative technical education at the undergraduate, graduate, and research levels. At present, the department offers an UG course (BE) with an intake of 360, a PG course in VLSI Design and Embedded Systems with an intake of 18 students and Ph.D. The department has got an excellent infrastructure with sophisticated labs, class rooms and R & D center. The department faculty and support personnel are dedicated towards helping students achieve success in today's world by offering them the knowledge and skills required. The department places a strong emphasis on leading-edge research through its research centre and research forum. These factors together enable our graduating students to create a big impact on the workforce from day one.

In addition to offering innovative courses, the department is more focused on improving students' educational experiences by offering value-added courses, skill-development programs, technical challenges and hackathons at state, national, and international arenas. Through its professional societies and technical clubs, the department takes great satisfaction in exposing its students to current industry developments and upcoming technologies. The dedicated staff at the ECE department help students procure jobs in prestigious companies both domestically and abroad.

Vision of the College:

To impart quality technical education with a focus on Research and Innovation emphasizing on Development of Sustainable and Inclusive Technology for the benefit of society.

Mission of the College:

- To provide an environment that enhances creativity and Innovation in pursuit of Excellence.
- To nurture teamwork in order to transform individuals as responsible leaders and entrepreneurs.
- To train the students to the changing technical scenario and make them to understand the importance of Sustainable and Inclusive technologies.

Vision of the Department

To achieve continuous improvement in quality technical education for global competence with focuson industry, societal needs, research and professional success.

Mission of the Department

- Offering quality education in Electronics and Communication Engineering with effective teaching learning process in multidisciplinary environment.
- Training the students to take-up projects in emerging technologies and work with team spirit.
- To imbibe professional ethics, development of skills and research culture for better placement opportunities.

Program Educational Objectives [PEOs]

The Graduate students must be able to:

PEO-1: Ready to apply the state-of-art technology in industry and meeting the societal needs with Knowledge of Electronics and Communication Engineering due to strong academic culture.

PEO-2: Competent in technical and soft skills to be employed with capability of working in Multidisciplinary domains.

PEO-3: Professionals, capable of pursuing higher studies in technical, research or management programs.

Programme Specific Outcomes [PSOs]

PSO-1: Design, develop and integrate electronic circuits and systems using current practices and Standards.

PSO-2: Apply knowledge of hardware and software in designing Embedded and Communication Systems.

Knowledge and Attitude Profile (WK) as per New NBA-2024

WK1: A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.

WK2: Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.

WK3: A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

WK4: Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.

WK5: Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.

WK6: Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.

WK7: Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.

WK8: Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.

WK9: Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

New POs as per NBA-2024

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of

experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

SDGs (Sustainable Development Goals) from UN

1. **No Poverty**: End poverty in all its forms everywhere
2. **Zero Hunger:** End hunger, achieve food security and improved nutrition and promote sustainable agriculture
3. **Good Health and Wellbeing:** Ensure healthy lives and promote well-being for all at all ages
4. **Quality Education:** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all
5. **Gender equality:** Achieve gender equality and empower all women and girls
6. **Clean water and Sanitation:** Ensure availability and sustainable management of water and sanitation for all
7. **Affordable and Clean Energy:** Ensure access to affordable, reliable, sustainable and modern energy for all

8. **Decent work and Economic growth:** Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all
9. **Industry, Innovation and Infrastructure:** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation
10. **Reduced Inequalities:** Reduce inequality within and among countries
11. **Sustainable cities and Communities:** Make cities and human settlements inclusive, safe, resilient and sustainable
12. **Responsible consumption and production:** Ensure sustainable consumption and production patterns
13. **Climate action:** Take urgent action to combat climate change and its impacts
14. **Life below water:** Conserve and sustainably use the oceans, seas and marine resources for sustainable development
15. **Life on Land:** Protect, restore and promote sustainable use of terrestrial ecosystems, sustainably manage forests, combat desertification, halt and reverse land degradation, and halt biodiversity loss
16. **Peace, justice and strong institutions:** Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels
17. **Partnership for Goals:** Strengthen the means of implementation and revitalize the global partnership for sustainable development

ADVANCED EMBEDDED SYSTEMS

Course Code	:	22EC72			**Credits**	:	04
L: T: P	:	3	0	2	**CIE Marks**	:	50
Exam Hours	:	03			**SEE Marks**	:	50
Total Hours	:	52			**CIE + SEE Marks**	:	100

Course Objectives

- Provide a comprehensive understanding of ARM Cortex-M processor architecture, its programmer's model, and system components for designing efficient embedded systems.
- Develop proficiency in ARM and Thumb instruction sets for implementing optimized assembly language programs.
- Equip students with the skills to configure and manage interrupts, exceptions, and low- power operational modes in Cortex-M processors.
- Enable students to analyze and configure memory hierarchy, cache systems, and memory protection mechanisms for secure and efficient embedded applications.
- Foster the ability to design and implement ARM-based embedded applications using network protocols and sustainable development principles for real-world problem solving.

Course Outcomes (COs)

1. Use the knowledge of ARM and Thumb instruction sets for developing software programs and applications.
2. Demonstrate the use of interrupts, exception handling, and low-power features of Cortex-M processors.
3. Analyse memory hierarchy, cache architecture, and memory protection mechanisms including MPU configuration in embedded systems.
4. Apply concepts of embedded network architectures and protocols to analyse Internet-enabled systems and sensor-based applications.
5. Collaborate in teams to develop ARM-based embedded system applications addressing sustainable development challenges and prepare comprehensive technical reports.

Module	Course Contents	Hours
1	**Introduction to ARM Cortex- M Processors:** Introduction, Advantages and Applications to Cortex_-M processors, Block diagram of Cortex m3 and M4 **Architecture:** Introduction to the architecture, Programmer's model, Behavior of the application program status register (APSR), Memory system, Exceptions and interrupts, System control block (SCB), Debug, Reset and reset sequence	8 CO1
2	**Introduction to ARM and Thumb Instruction Set:** Moving data within the processor, Memory access instructions, Arithmetic operations, Logic operations, Shift and rotate instructions, Data conversion operations (extend and reverse ordering), Bit-field processing instructions, Compare and test, Program flow control, Saturation operations, Exception-related instructions, Sleep mode-related instructions, Memory barrier instructions. **Low Power and System Control Features** Low power system requirements, Low power characteristics of the Cortex_- M3 and Cortex-M4 processors, Low power features.	8 CO1,2
3	**Exceptions and Interrupts:** Overview of exceptions and interrupts, Exception types, Exception sequence overview, Details of NVIC registers for interrupt control, Details of special registers for exception or interrupt masking. **The SysTick Timer**-Why have a SysTick timer? Operations of the SysTick timer Using the SysTick timer.	8 CO2
4	**Memory protection unit:** Overview of the MPU, MPU registers, Setting up the MPU. **Caches and memory management:** The Memory Hierarchy and Cache Memory, Cache architecture (Basic architecture, operation, relationship between main memory and cache).	8 CO3
5	**Networks for Embedded systems:** Distributed embedded architectures, The I2C Bus, Ethernet, Internet-enabled systems: Internet, Internet Applications, Internet Security, Sensor networks, Design Example: Elevator Controller	8 CO4

List of Experiments in IPCC Course Integrated Lab USING ARM Cortex M3 (LPC1768)

Expt. No.	Contents of the Experiments (2-3 expts from each module / unit)	Hrs.
	Module 1 and 2	2hrs
1.	ALP Programs based on Load store instructions	
2.	ALP Programs based on Arithmetic Operations	
3.	C Program to control the status of an LED through a Simple Switch	
	Module 3	2hrs
4.	Program to illustrate the concepts of Memory Protection Unit	
5.	Programs related to Interrupts and exception handling	
6.	Programs related to Low Power Modes in ARM Cortex M3	
	Module 4	2hrs
7.	Programs illustrating the use of Special registers	
8.	Program to illustrate the concepts of SysTick Timer	
9.	Program to understand Two stack model in ARM	
	Module 5	2hrs
10.	Program to illustrate UART.	
11.	Program to illustrate I2C .	
12.	Program to implement FIR Filter	

Sl no	Experiment name	Hours	COs
1.	Write an ALP to demonstrate LOAD-STORE ARCHITECTURE	02	CO1,2
2.	Write an ALP to perform 32-bit addition, subtraction, multiplication, and division on operands stored in RAM locations.	02	CO1,2
3.	Write an embedded C program to interface a simple switch and display its status through LED.	02	CO1,2
4.	Write a C program to demonstrate the working of Memory Protection Unit (MPU).	02	CO3
5.	Write a C program to demonstrate the use of the SysTick timer interrupt managed by Nested Vector Interrupt Controller(NVIC).	02	CO2
6.	Write an embedded C program to generate a sine wave using SysTick timer in LPC1768.	02	CO2
7.	Write a C program for ARM Cortex-M3 (LPC1768) to demonstrate the working of low power modes: Idle, Sleep, Deep Sleep, and Power Down.	02	CO1,2
8.	Write an ARM C program for the LPC1768 microcontroller to initialize UART0 for serial communication at 9600 baud rate and send a string repeatedly in an infinite loop with a simple software delay between transmissions.	02	CO4
9.	Write an embedded C program to demonstrate the working of Inter Integrated Circuit (I^2C) using LPC1768 and verify the same in Keil simulator.	02	CO4
10.	Write a C program for the ARM Cortex-M3 (LPC1768) microcontroller to demonstrate the working of special interrupt mask registers — PRIMASK, BASEPRI, and FAULTMASK.	02	CO3
11.	Write an embedded C program to demonstrate the working of two-stack model in ARM- LPC1768.	02	CO1,2
12.	Write an embedded C program to implement Type-II Linear FIR Filter using ARM LPC1768.	02	CO4

DEPARTMENT of ELECTRONICS AND COMMUNICATION ENGINEERING

Advanced Embedded System Laboratory

Index Sheet

Sl no	Program	Date of Conduction	Marks obtained					Sign
			C (10)	A (10)	R (5)	V (5)	Total	

DAYANANDA SAGAR COLLEGE OF ENGINEERING

DEPARTMENT OF ELCTRONICS & COMMUNICATION ENGINEERING

DO's

- All the students should come to LAB on time with proper dress code and identity cards.
- Keep your belongings in the bagrack of laboratory.
- Students have to enter their name, USN, time-in/out and signature in the log register maintained in the laboratory.
- All the students should submit their records before the commencement of Laboratory experiments.
- Students should come to the lab well prepared for the experiments which are to be performed in that particular session.
- Students are asked to do the experiments on their own and should not waste their precious time by talking, roaming and sitting idle in the labs.
- Observation book and record book should be complete in all respects and it should be corrected by the staff member.
- Before leaving the laboratory students should arrange their chairs and leave in orderly manner after completion of their scheduled time.
- Prior permission to be taken, if for some reasons, they cannot attend lab.
- Immediately report any sparks/ accidents/ injuries/ any other untoward incident to the faculty /instructor.
- In case of an emergency or accident, follow the safety procedure.
- Switch OFF the power supply after completion of experiment

DONT's

- The use of mobile/ any other personal electronic gadgets is prohibited in the laboratory.
- Do not make noise in the Laboratory & do not sit on experiment table.
- Do not make loose connections and avoid overlapping of wires
- Don't switch on power supply without prior permission from the concerned staff.
- Never leave the experiments while in progress.
- Do not leave the Laboratory without the signature of the concerned staff in observation book.

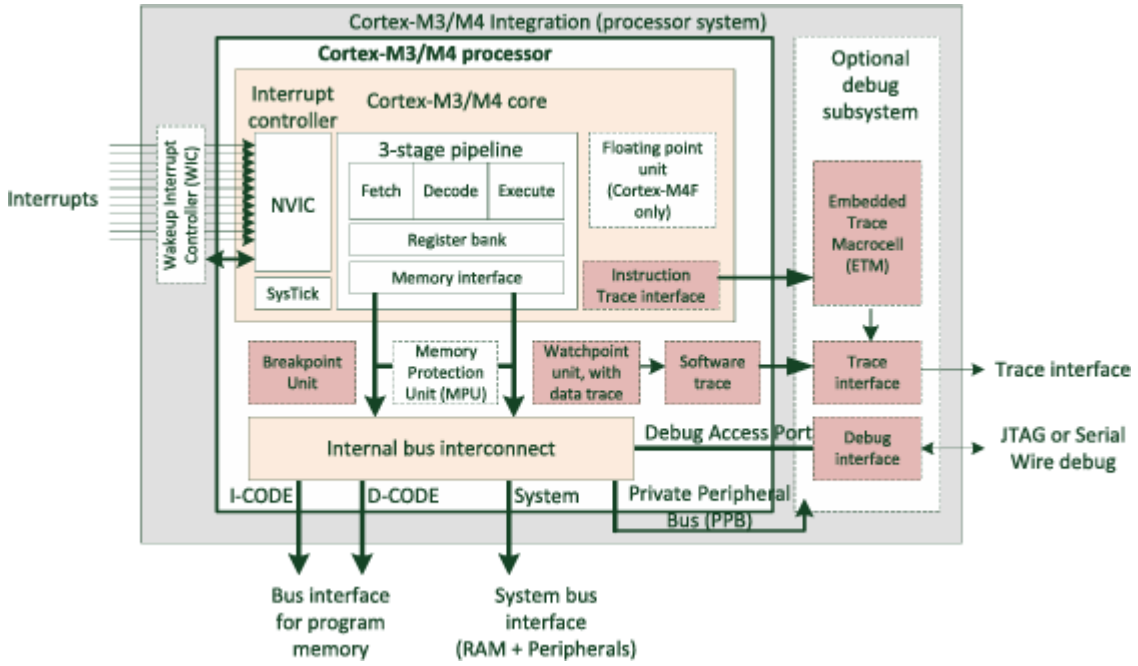
INTRODUCTION TO ARM CORTEX M3

The ARM Cortex™-M3 processor, the first of the Cortex generation of processors released by ARM in 2006, was primarily designed to target the 32-bit microcontroller market. The Cortex-M3 processor provides excellent performance at low gate count and comes with many new features previously available only in high-end processors.

ARCHITECTURE OF ARM CORTEX M3

The Cortex™-M3 is a 32-bit microprocessor. It has a 32-bit data path, a 32-bit register bank, and 32- bit memory interfaces .The processor has a Harvard architecture, which means that it has a separate instruction bus and data bus. This allows instructions and data accesses to take place at the same time, and as a result of this, the performance of the processor increases because data accesses do not affect the instruction pipeline. This feature results in multiple bus interfaces on Cortex-M3, each with optimized usage and the ability to be used simultaneously. However, the instruction and data buses share the same memory space (a unified memory system). In other words, you cannot get 8 GB of memory space just because you have separate bus interfaces.

For complex applications that require more memory system features, the Cortex-M3 processor has an optional Memory Protection Unit (MPU), and it is possible to use an external cache if it's required. Both little endian and big endian memory systems are supported. The Cortex-M3 processor includes a number of fixed internal debugging components. These components provide debugging operation supports and features, such as breakpoints and watch points.



REGISTERS:

The Cortex-M3 processor has registers R0 through R15, R13 (the stack pointer) is banked, with only one copy of the R13 visible at a time.

1. R0–R12: General-Purpose Registers R0–R12 are 32-bit general-purpose registers for data operations. Some 16-bit Thumb® instructions can only access a subset of these registers (low registers, R0–R7)
2. R13: Stack Pointers the Cortex-M3 contains two stack pointers (R13). They are banked so that only one is visible at a time. The two stack pointers are as follows:
    a. Main Stack Pointer (MSP): The default stack pointer, used by the operating system (OS) kernel and exception handlers
    b. Process Stack Pointer (PSP): Used by user application code the lowest 2 bits of the stack pointers are always 0, which means they are always word aligned.
3. R14: The Link Register When a subroutine is called, the return address is stored in the link register.
4. R15: The Program Counter The program counter is the current program address. This register can be written to control the program flow.

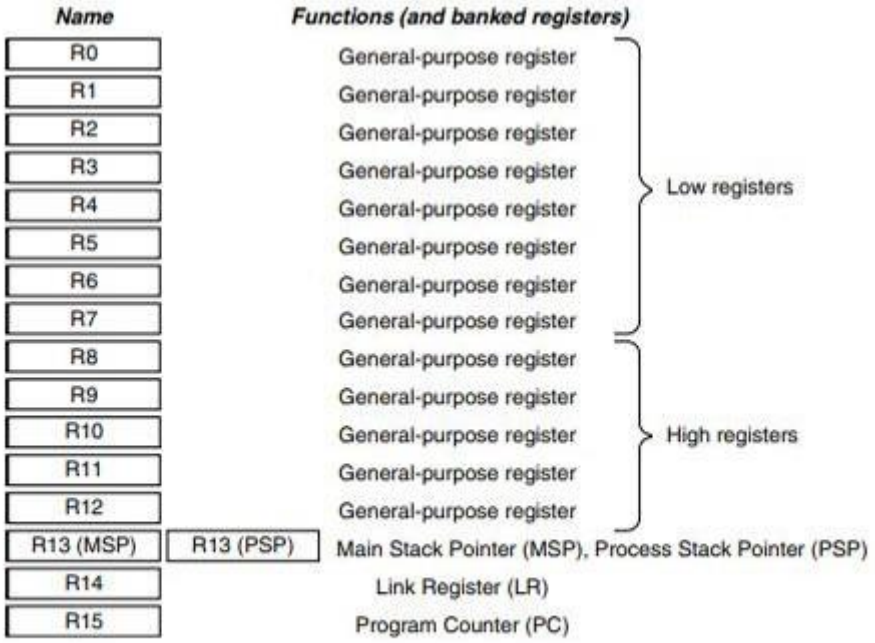


Fig 2: General purpose registers in Cortex M3

SPECIAL REGISTERS:

The Cortex-M3 processor also has a number of special registers. They are as follows:

• Program Status registers (PSRs)

• Interrupt Mask registers (PRIMASK, FAULTMASK, and BASEPRI)

• Control register (CONTROL) These registers have special functions and can be accessed only by special instructions

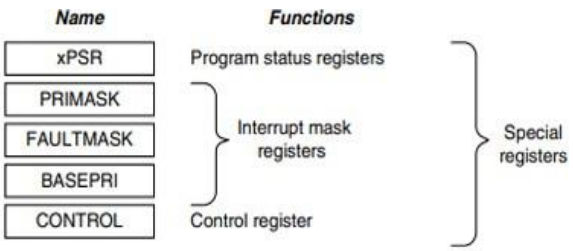


Fig 3: Special Registers

OPERATION MODES

The Cortex-M3 processor has two modes and two privilege levels. The operation modes (thread mode and handler mode) determine whether the processor is running a normal program or running an exception handler like an interrupt handler or system exception handler (see Figure 2.4). The privilege levels (privileged level and user level) provide a mechanism for safeguarding memory accesses to critical regions as well as providing a basic security model.

When the processor is running a main program (thread mode), it can be either in a privileged state or a user state, but exception handlers can only be in a privileged state. When the processor exits reset, it is in thread mode, with privileged access rights. In the privileged state, a program has access to all memory ranges (except when prohibited by MPU settings) and can use all supported instructions.

Software in the privileged access level can switch the program into the user access level using the control register. When an exception takes place, the processor will always switch back to the privileged state and return to the previous state when exiting the exception handler. A user program cannot change back to the privileged state by writing to the control register. It has to go through an exception handler that programs the control register to switch the processor back into the privileged access level when returning to thread mode. The separation of privilege and user levels improves system reliability by preventing system configuration registers from being accessed or changed by some untrusted programs.

	Privileged	*User*
When running an exception handler	Handler mode	
When not running an exception handler (e.g., main program)	Thread mode	Thread mode

Fig 4 : Operation Modes and Privilege Levels in Cortex-M3.

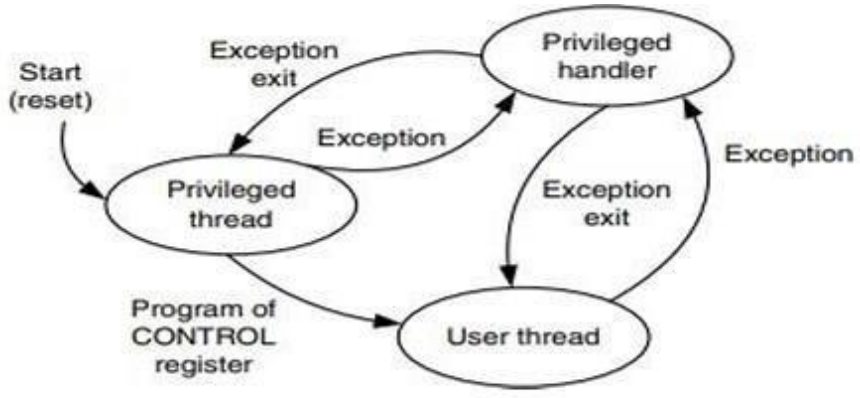
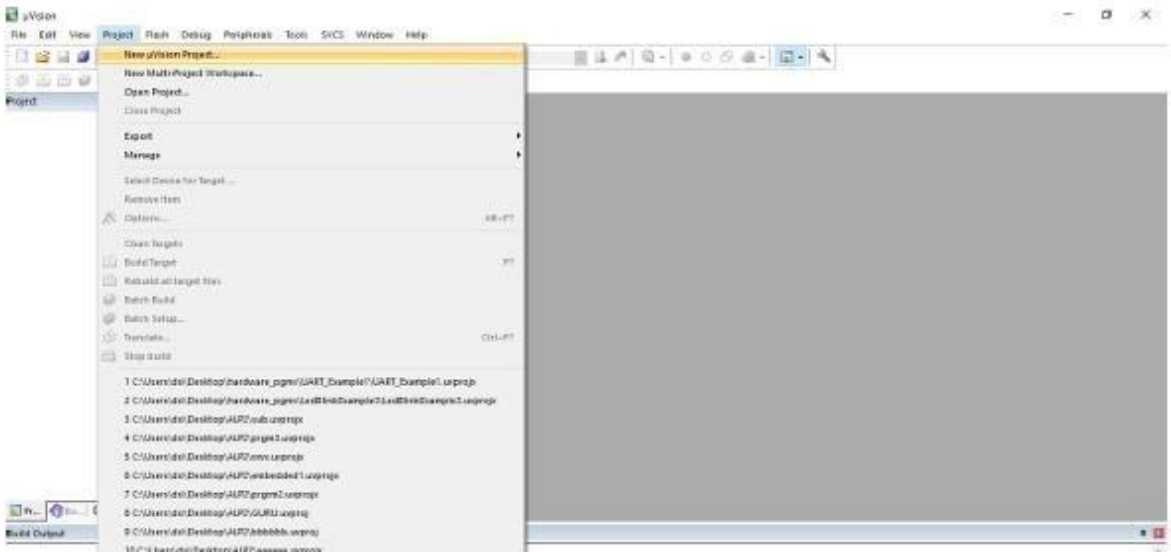


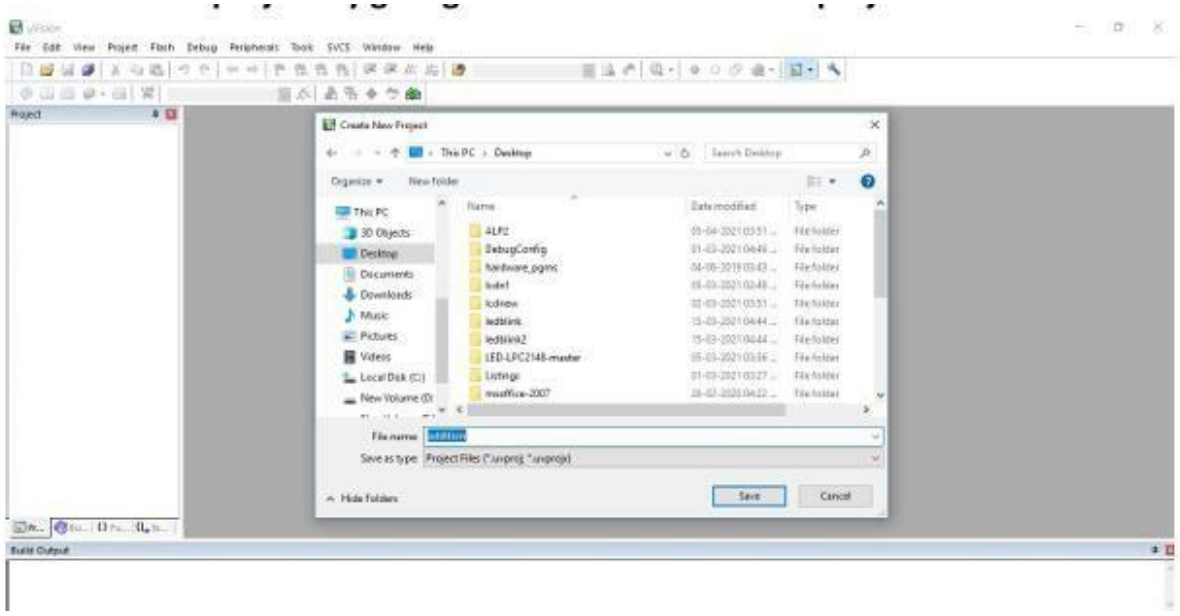
Fig 5: Allowed Operation Mode Transitions

PROCEDURE FOR EXECUTING ALP

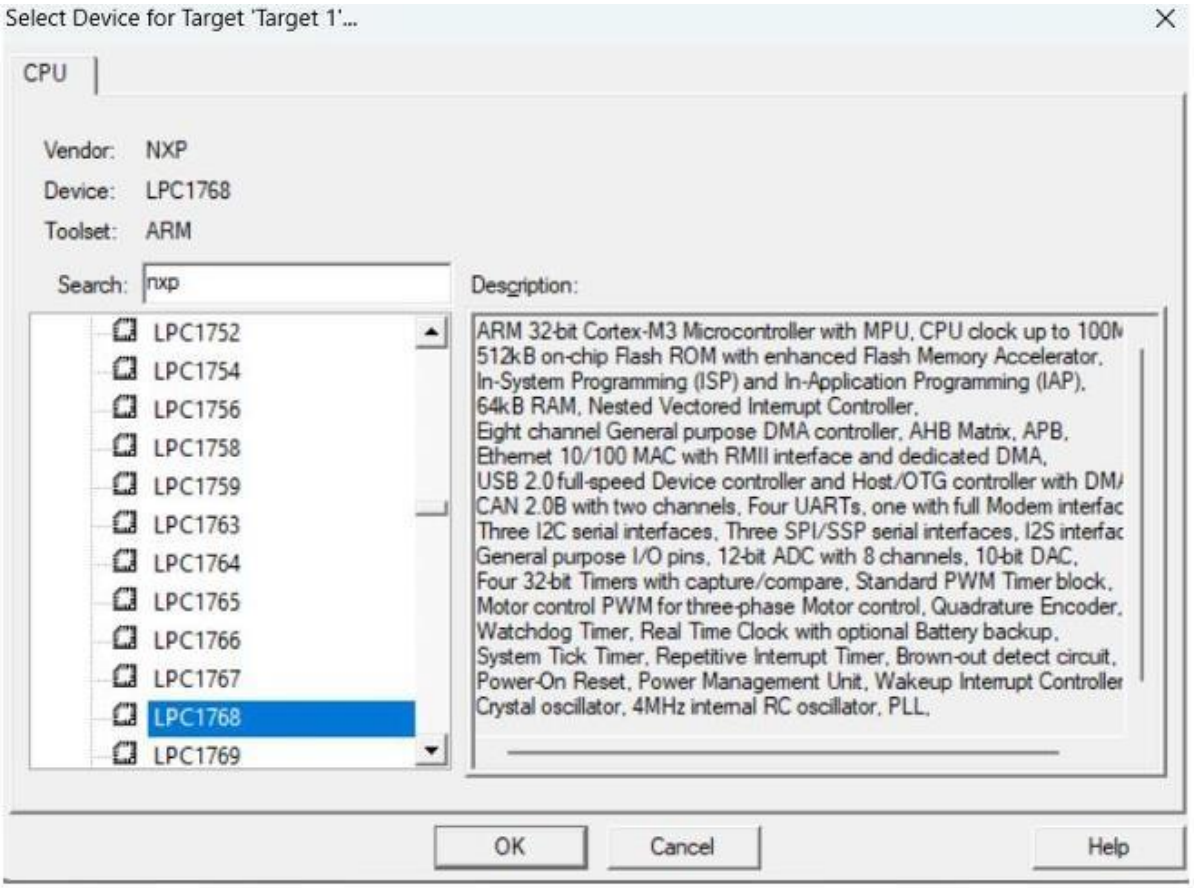
1. **Open Keil Microvision4 on the desktop.**
2. **Go to project and click on create new project.**



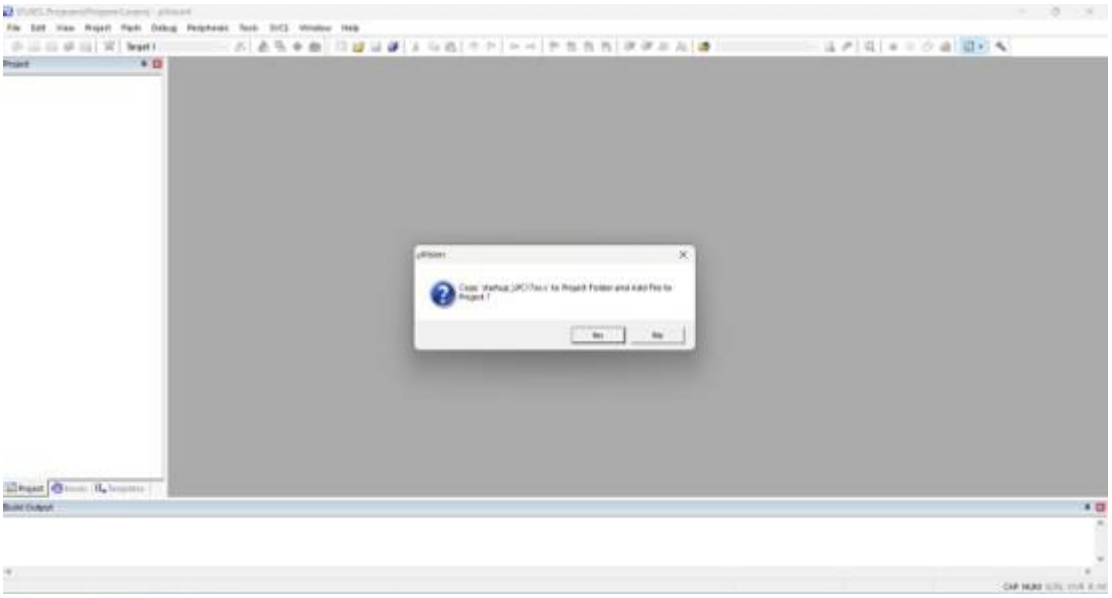
3. **Create a new project by giving the name and save the project**



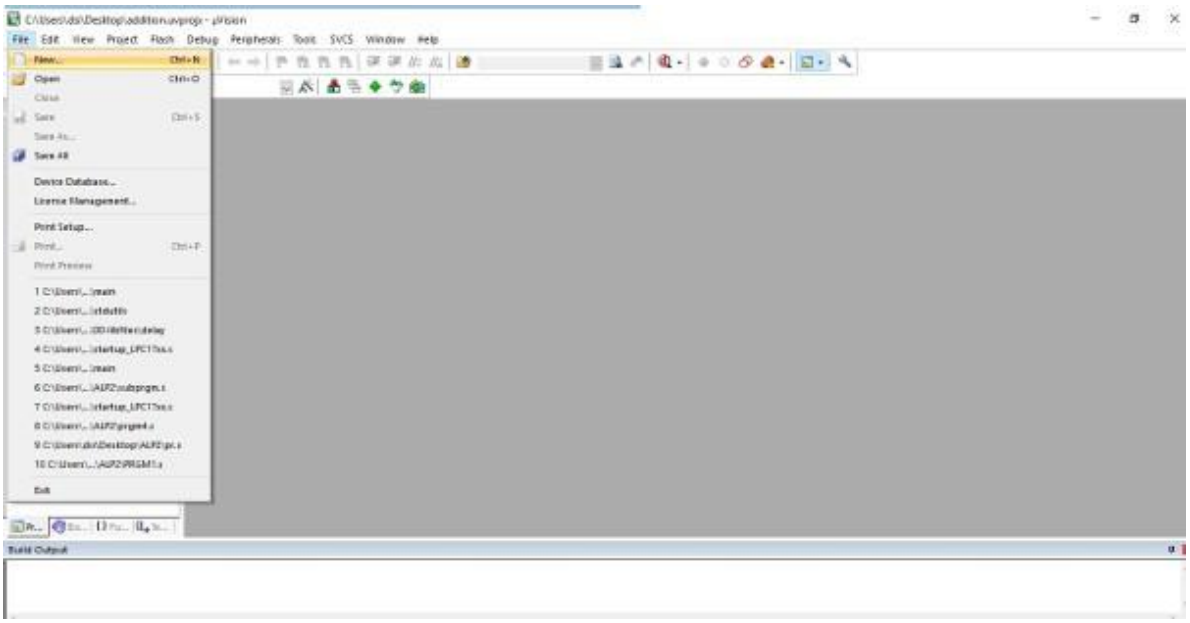
4. **In Select Device for Target, select software packs- NXP- LPC17XX series- LPC1768.**



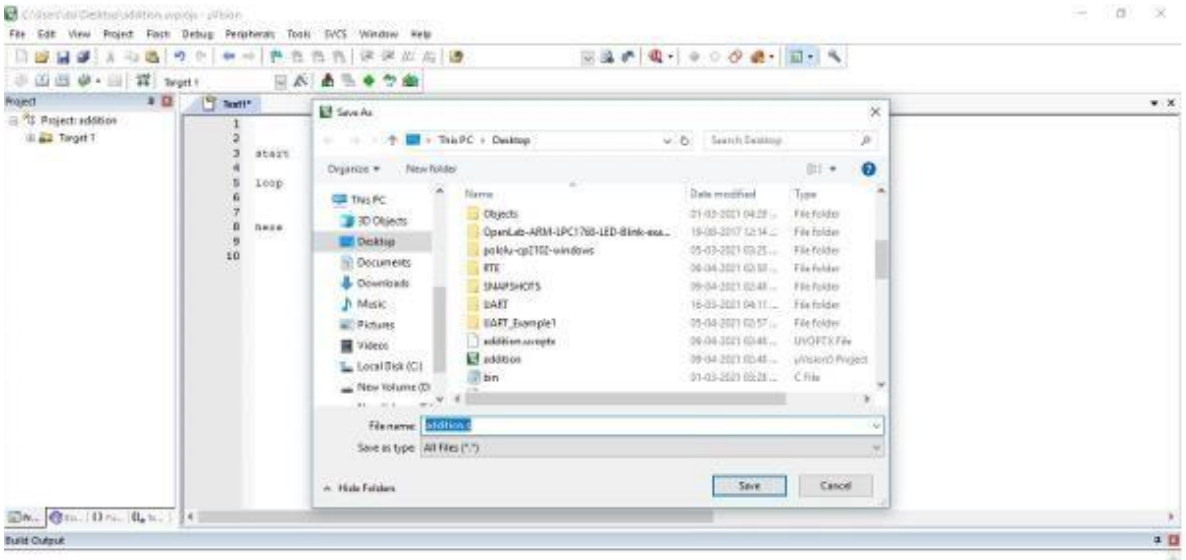
5. **Copy startup file-- .Click OK.**



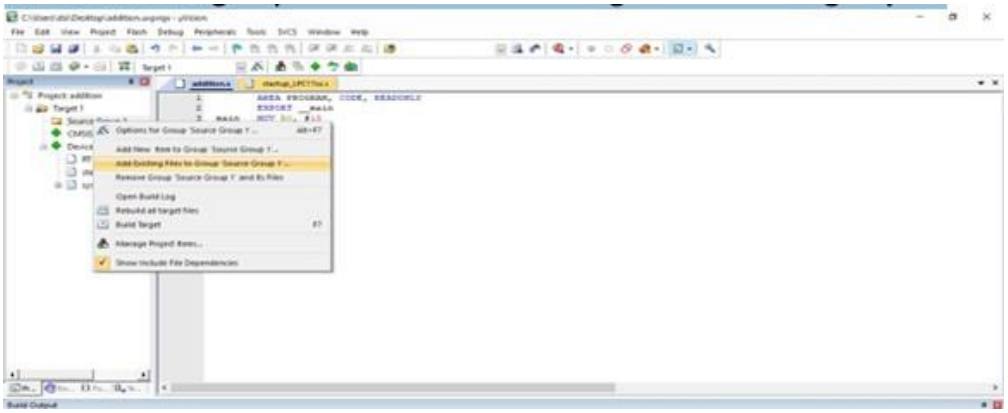
6. Click on File—New and type the program.



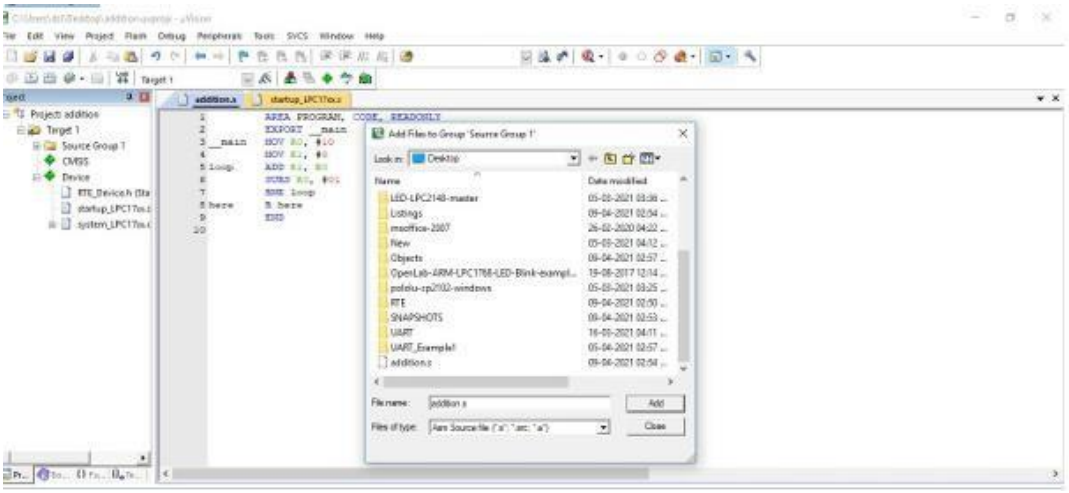
7. After typing the program save the file with .s or .asm extension



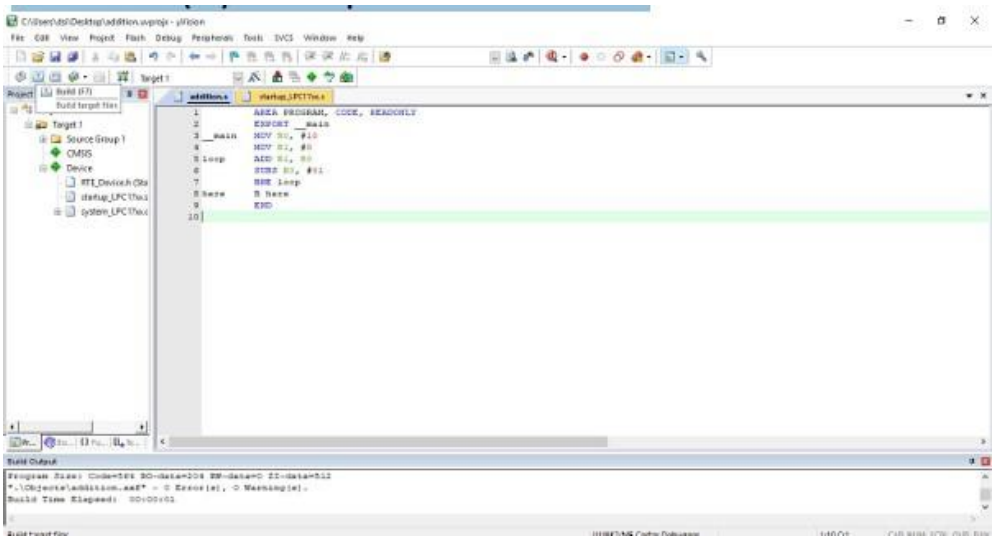
8. **Click on Source group and select " Add existing files to source group 1"**



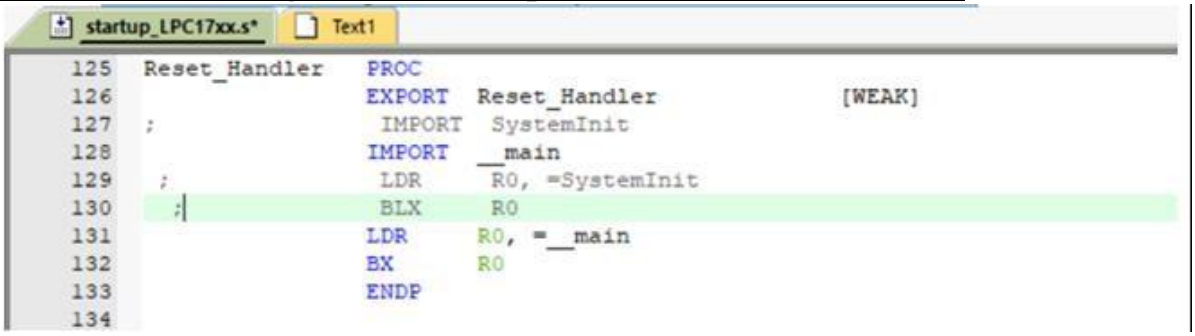
9. **Select the existing asm file from the folder and add it to the source group.**



10. **Click on Build(F7) and compile the code for errors.**

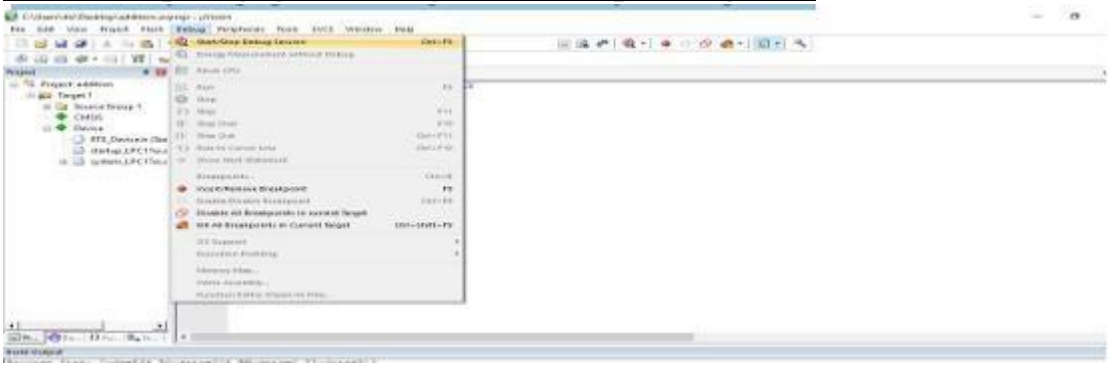


11. **Comment following lines in startup file— 127,128,129,272,273**

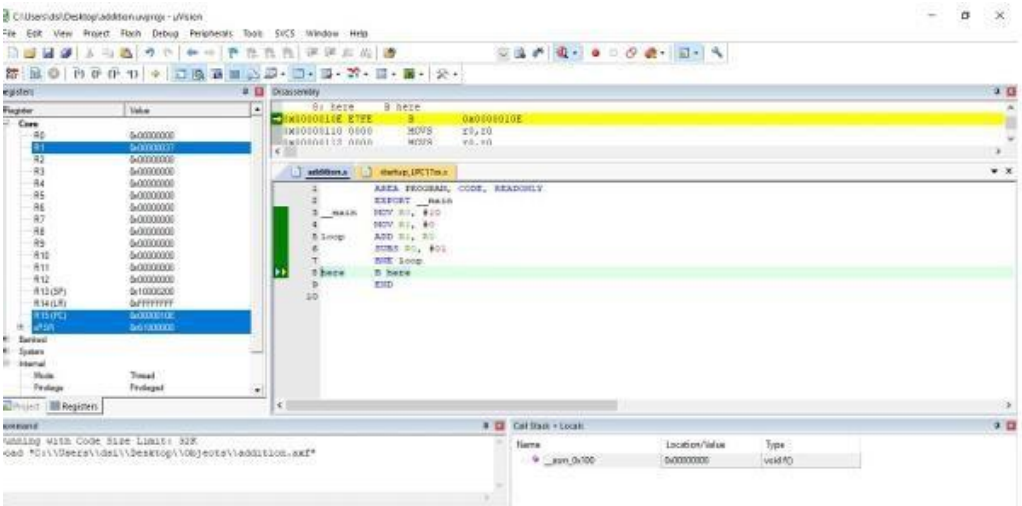




12. **After compiling, goto debug—start/stop debug—ok**



13. **In the debug window select run(F5) or Step one line(F11) to see the output in registers.**



Experiment 1

AIM: **Write an ALP to demonstrate LOAD-STORE ARCHITECTURE**

PROGRAM:

```
        AREA    LOAD_STORE_DEMO, CODE, READONLY
        EXPORT __main

__main      LDR    R0, =SRC_AREA      ; Base address for source
        LDR    R1, =DST_AREA      ; Base address for destination

        ;============================
        ; STR (Store Word) and LDR (Load Word)
        ;============================
        LDR    R2, =0x12345678
        STR    R2, [R1]           ; Store word at DST_AREA
        LDR    R3, [R1]           ; Load word back

        ;============================
        ; STRB (Store Byte) and LDRB (Load Byte)
        ;============================
        LDR    R4, =0xABCDEF10
        STRB   R4, [R1, #4]       ; Store byte at DST_AREA+4
        LDRB   R5, [R1, #4]       ; Load byte back to R5

        ;============================
        ; STRH (Store Halfword) and LDRH (Load Halfword)
        ;============================
        LDR    R6, =0xCDEFABCD
        STRH   R6, [R1, #8]       ; Store halfword at DST_AREA+8
        LDRH   R7, [R1, #8]       ; Load halfword back to R7

        ;============================
        ; STM and LDM (Store and Load Multiple)
        ;============================
        LDR    R8, =0xAAAA1111
        LDR    R9, =0xBBBB2222
        LDR    R10, =0xCCCC3333

    STMIA  R1!, {R8-R10}      ; Store R8, R9, R10 to DST_AREA and update R1
    SUB    R1, R1, #12        ; Restore original address

    MOV    R8, #0             ; Clear R8, R9, R10
    MOV    R9, #0
    MOV    R10, #0

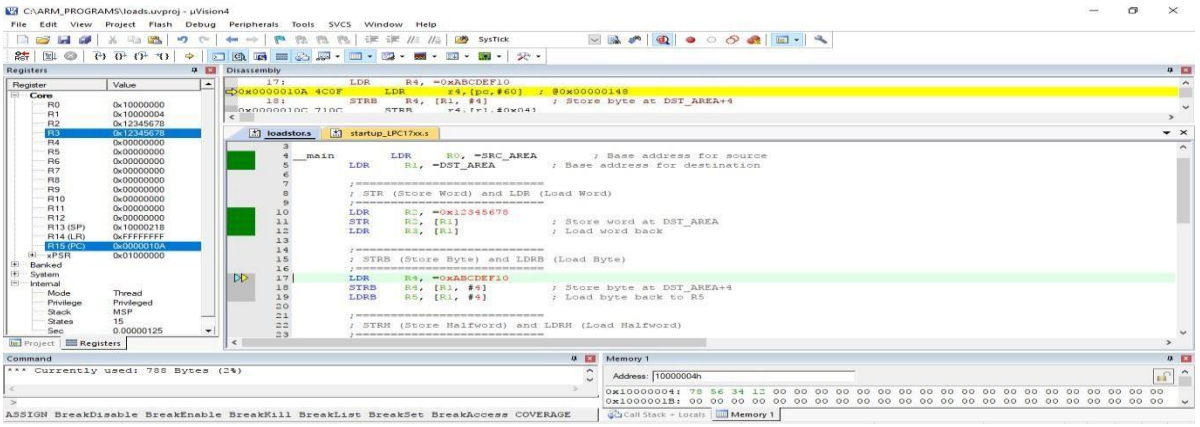
    LDMIA  R1!, {R8-R10}      ; Load back values to R8, R9, R10

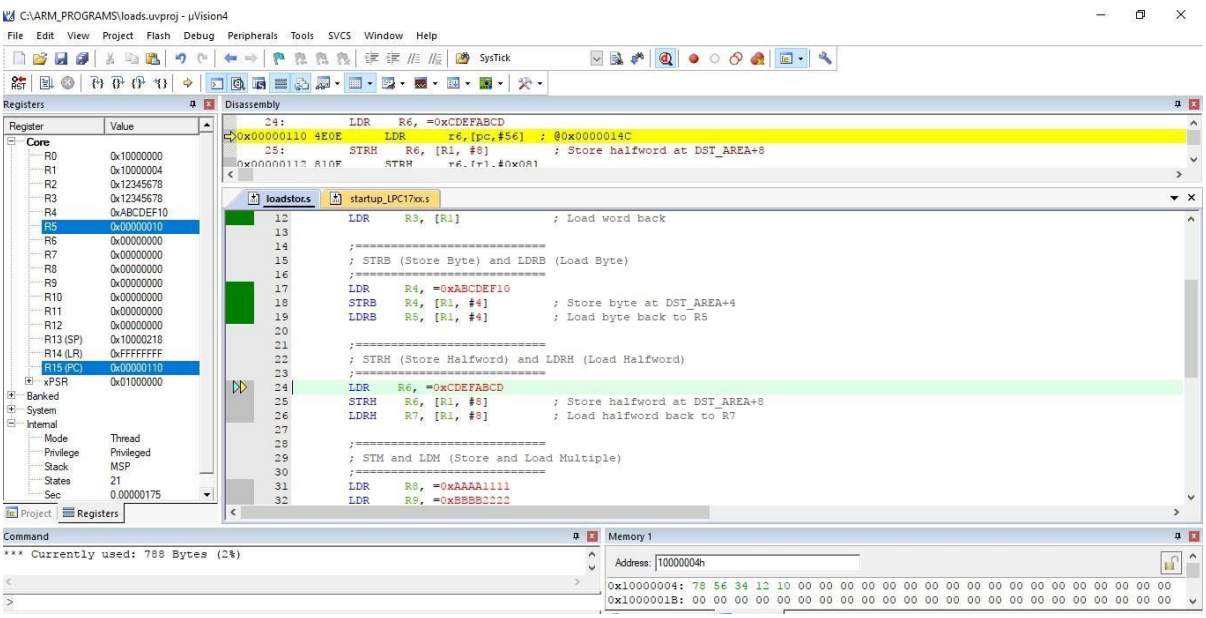
STOP   B      STOP             ; Infinite loop

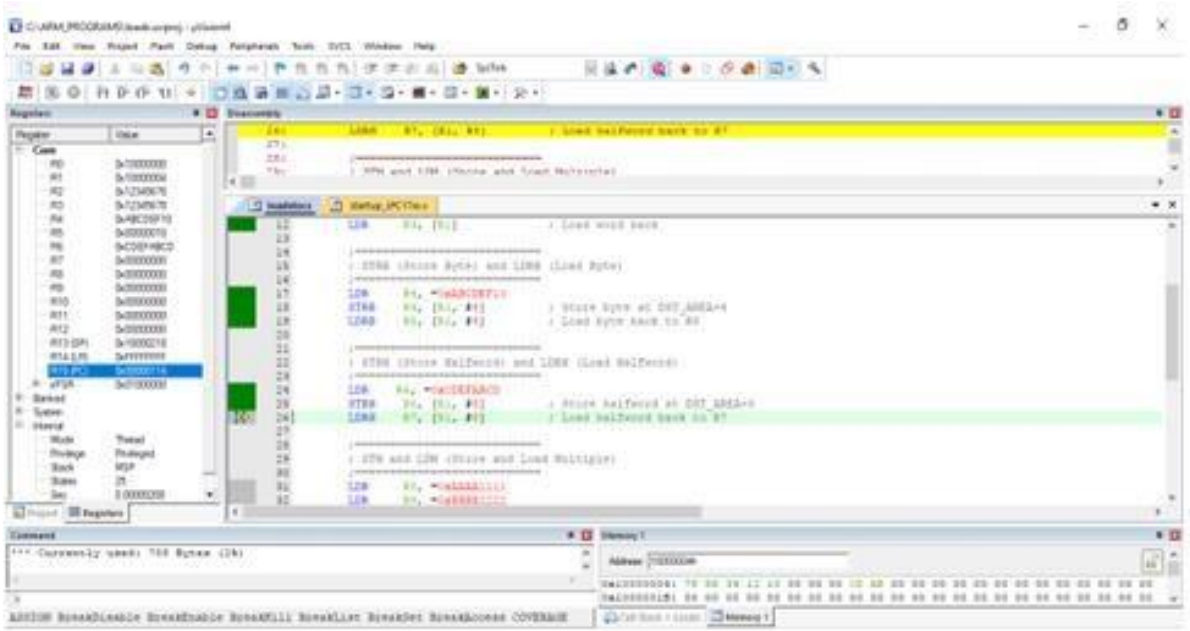
;============================
; Data Section
;============================
    AREA   DATA, DATA, READWRITE
SRC_AREA   DCD    0x11223344     ; Source area (not used here but can be)
DST_AREA   DCD    0x0, 0x0, 0x0, 0x0 ; Destination area for STR, STRB, STRH, STM

    END
```

RESULT:


C:\ARM_PROGRAMS\loads.uvproj - µVision4
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Register Value
Core
R0 0x10000000
R1 0x10000004
R2 0x12345678
R3 0x12345678
R4 0x00000000
R5 0x00000000
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000000
R13 (SP) 0x10000218
R14 (LR) 0xFFFFFFFF
R15 (PC) 0x0000010A
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 15
Sec 0.00000125
Disassembly
0x0000010A 4C0F LDR R4,[pc,#60] ; @0x00000148
STRB R4, [R1, #4] ; Store byte at DST_AREA+4
loadstor.s
startup_LPC17xx.s
__main LDR R0, =SRC_AREA ; Base address for source
LDR R1, =DST_AREA ; Base address for destination
; STR (Store Word) and LDR (Load Word)
LDR R2, =0x12345678
STR R2, [R1] ; Store word at DST_AREA
LDR R3, [R1] ; Load word back
; STRB (Store Byte) and LDRB (Load Byte)
LDR R4, =0xABCDEF10
STRB R4, [R1, #4] ; Store byte at DST_AREA+4
LDRB R5, [R1, #4] ; Load byte back to R5
; STRH (Store Halfword) and LDRH (Load Halfword)
Command
*** Currently used: 788 Bytes (2%)
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE
Memory 1
Address: 10000004h
0x10000004: 78 56 34 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000001B: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00


C:\ARM_PROGRAMS\loads.uvproj - µVision4
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Register Value
Core
R0 0x10000000
R1 0x10000004
R2 0x12345678
R3 0x12345678
R4 0xABCDEF10
R5 0x00000010
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000000
R13 (SP) 0x10000218
R14 (LR) 0xFFFFFFFF
R15 (PC) 0x00000110
xPSR 0x01000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 21
Sec 0.00000175
Disassembly
0x00000110 4E0E LDR r6,[pc,#56] ; @0x0000014C
STRH R6, [R1, #8] ; Store halfword at DST_AREA+8
loadstor.s
startup_LPC17xx.s
LDR R3, [R1] ; Load word back
; STRB (Store Byte) and LDRB (Load Byte)
LDR R4, =0xABCDEF10
STRB R4, [R1, #4] ; Store byte at DST_AREA+4
LDRB R5, [R1, #4] ; Load byte back to R5
; STRH (Store Halfword) and LDRH (Load Halfword)
LDR R6, =0xCDEFABCD
STRH R6, [R1, #8] ; Store halfword at DST_AREA+8
LDRH R7, [R1, #8] ; Load halfword back to R7
; STM and LDM (Store and Load Multiple)
LDR R8, =0xAAAA1111
LDR R9, =0xBBBB2222
Command
*** Currently used: 788 Bytes (2%)
Memory 1
Address: 10000004h
0x10000004: 78 56 34 12 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x1000001B: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00



Experiment 2

AIM: **Write an ALP to perform 32-bit addition, subtraction, multiplication, and division on operands stored in RAM locations.**

PROGRAM:

```
            AREA PROGRAM, CODE , READONLY

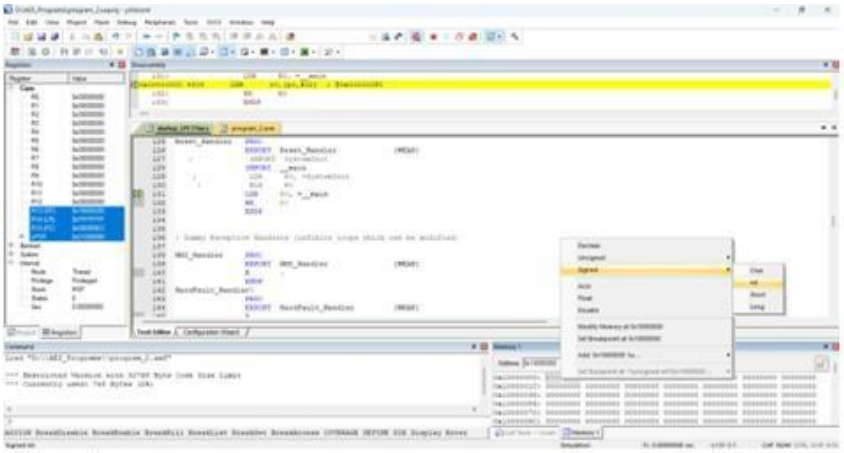
            EXPORT __main

__main      LDR R1, = 0x10000000 ;memory location for condition
            LDR R2, [R1]
            LDR R3, [R1, #4] ;Operand 1
            LDR R4, [R1, #8] ;Operand 2
            CMP R2, #1 ;1: Addition
            BNE CHK_SUB
            ADDS R7, R3, R4 ;R7=R3+R4
            B LAST
CHK_SUB     CMP R2, #2 ;2: subtraction
            BNE CHK_MUL
            SUBS R7, R3, R4 ;R7=R3-R4
            B LAST
CHK_MUL     CMP R2, #3 ; 3: Multiplication
            BNE CHK_DIV
            UMULL R7, R8, R3, R4 ; [RdHi,RdLow]=Rn*Rm
            B LAST ; UMULL:Unsigned multiply long
CHK_DIV     CMP R2, #4      ; 4: Division
            BNE LAST
            UDIV R7, R3, R4   ;R7=R3/R4

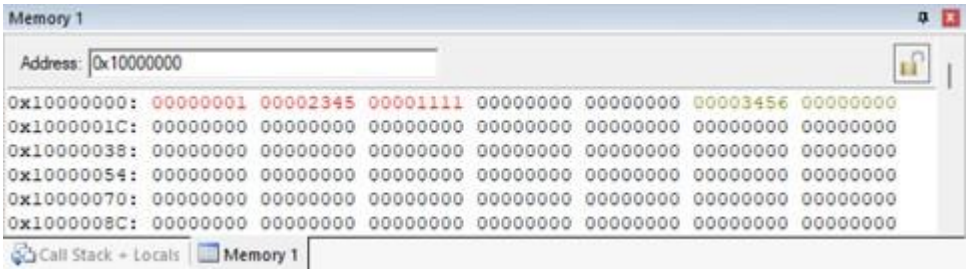
LAST        STR R7, [R1, #20]     ;store result in memory location 0x10000014
            STR R8, [R1, #24]     ;store the RdLow in memory location ;0x10000018 for multiplication.
STOP        B STOP
      END
```

RESULT:

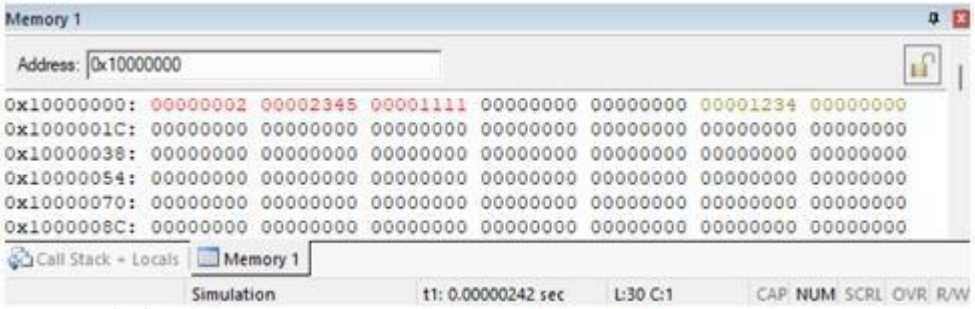
In debug session, open memory window and type address:0x10000000. Right click on the content of any location and select unsigned -> int. At RAM location 0x10000000, enter the condition code as per the table below.



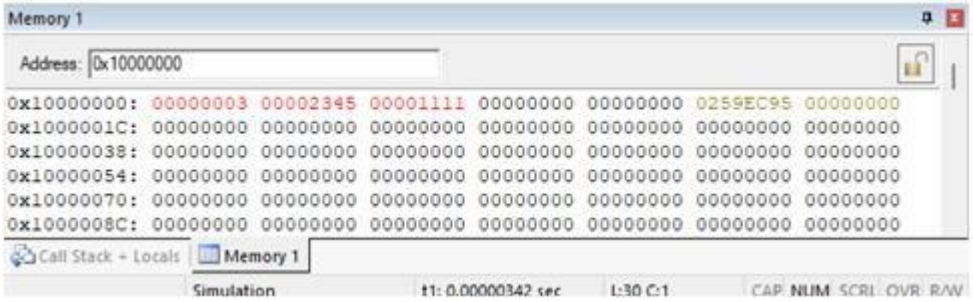
Case 1: Addition



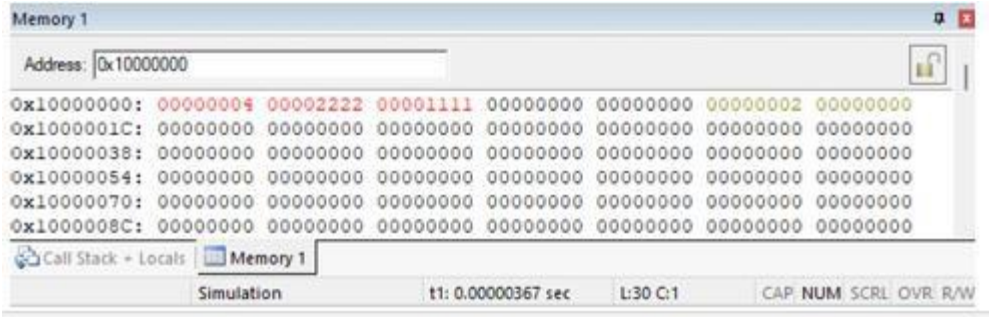
Case 2: Subtraction



Case 1: Multiplication



Case 1: Division



Experiment 3

AIM: **Write an embedded C program to interface a simple switch and display its status through LED.**

Theory: LPC1768 has its GPIOs divided into five ports PORT0 - PORT4. The Below registers will be used for Configuring and using the GPIOs registers for sending and receiving the Digital signals.

LPC1768 GPIO	Description
PINSEL	Selects the pin function
FIODIR	Configures pin direction
FIOSET, FIOCLR, FIOPIN	Access the port pin

PINSEL: GPIO Pins Select Register(32-Bit)

Almost all the LPC1768 pins are multiplexed to support more than 1 function. Every GPIO pin has a minimum of one function and max of four functions. The required function can be selected by configuring the PINSEL register. As there can be up to 4 functions associated with a GPIO pin, two bits for each pin are available to select the function. This implies that we need two PINSEL registers to configure a PORT pins. By this the first 16(P0.0-P0.15) pin functions of PORT0 can be selected by 32 bits of PINSELO register. The remaining 16 bits(P0.16-P0.32) are configured using 32bits of PINSEL1 register. As mentioned earlier every pin has max of four functions. Below table shows how to select the function for a particular pin using two bits of the PINSEL register.

Value (Binary)	Function Description	PINSEL Enumeration
00	Primary (default) function – typically GPIO port	PINSEL_FUNC_0
01	First alternate function	PINSEL_FUNC_1
10	Second alternate function	PINSEL_FUNC_2
11	Third alternate function	PINSEL_FUNC_3

FIODIR: Fast GPIO Direction Control Register. (32-Bit)

This register individually controls the direction of each port pin.

Value	GPIO Direction
0	Input
1	Output

FIOSET: Fast Port Output Set Register. (32-Bit)

This register controls the state of output pins. Writing 1s produces highs at the corresponding port pins. Writing 0s has no effect. Reading this register returns the current contents of the port output register not the physical port value.

Value	FIOSET Action
0	No effect
1	Set pin output HIGH

FIO0SET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	0	0	0	0	1	1

FIOCLR:Fast Port Output Clear Register. (32-Bit)

This register controls the state of output pins. Writing 1s produces lows at the corresponding port pins. Writing 0s has no effect.

Value	FIOCLR Action
0	No effect
1	Set pin output LOW

FIO0CLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	1	0	0	0	1	1

FIOPIN: Fast Port Pin Value Register. (32-Bit)

This register is used for both reading and writing data from/to the PORT.

Output: Writing to this register places corresponding values in all bits of the particular PORT pins.

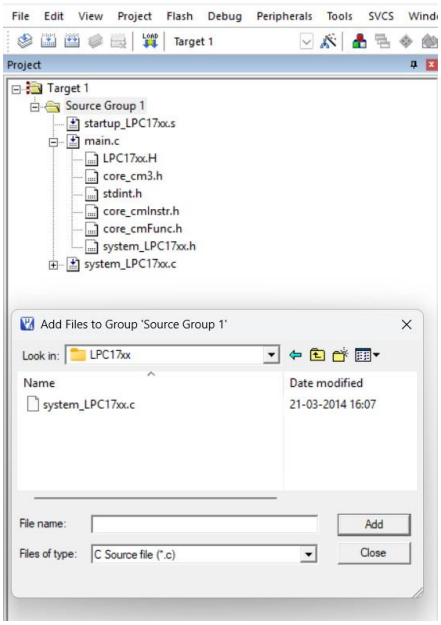
Input: The current state of digital port pins can be read from this register, regardless of pin direction or alternate function selection .

FIO0PIN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	1	0	1	0	0	1	1	1	1	0	0	0	0

STEPS TO EXECUTE:

1. Same steps to be followed from experiment 1, additional steps for embedded C programs are mentioned below.
2. Save the file with .c extension and add to the source group.
3. Goto → Keil folder(C drive) → ARM→ Startup → NXP → LPC17xx → Select system_LPC17xx.c and add to source group.



4. After Build and Debug , check the output in Peripherals → Fast GPIO → Port 0 for output and Port 2 for input.

PROGRAM

```
#include<LPC17xx.H>
int main(void)
{
      SystemInit();
         SystemCoreClockUpdate();
         LPC_PINCON->PINSEL1=0X00000000;
         LPC_GPIO0->FIODIR|=0X30000000;
while(1)
     {
          if(!(LPC_GPIO2->FIOPIN&0X00000800))
               {
                LPC_GPIO0->FIOSET=0X03000000;
               }
               else
               {
                LPC_GPIO0->FIOCLR=0X03000000;
               }
     }
}
```

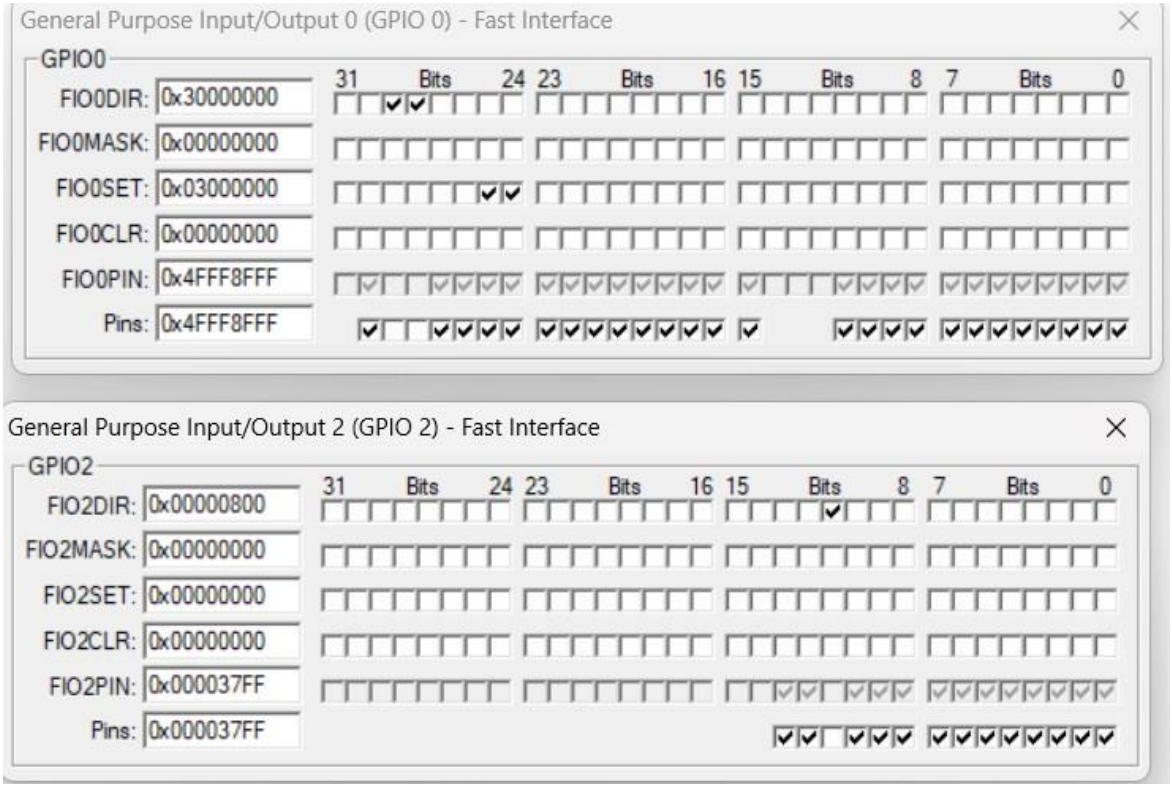
Understand the code:

- PINSEL1 = 0x00000000 → Configures pins P0.16 to P0.31 as GPIO.
- FIODIR |= 0x30000000 → Sets P0.28 & P0.29 as outputs.
- if (!(P2.11)) [FIOPIN&0X00000800] → Checks if switch on P2.11 is pressed (active low).
- If pressed → [FIOSET=0X03000000] set P0.24 & P0.25 high (turn LEDs ON).

  Else → clear P0.24 & P0.25 (turn LEDs OFF).

RESULT:

- If P2.11 = 0 → LPC_GPIO0->FIOSET sets P0.28 & P0.29 HIGH.
- If P2.11 = 1 → LPC_GPIO0->FIOCLR sets them LOW.



Experiment 4

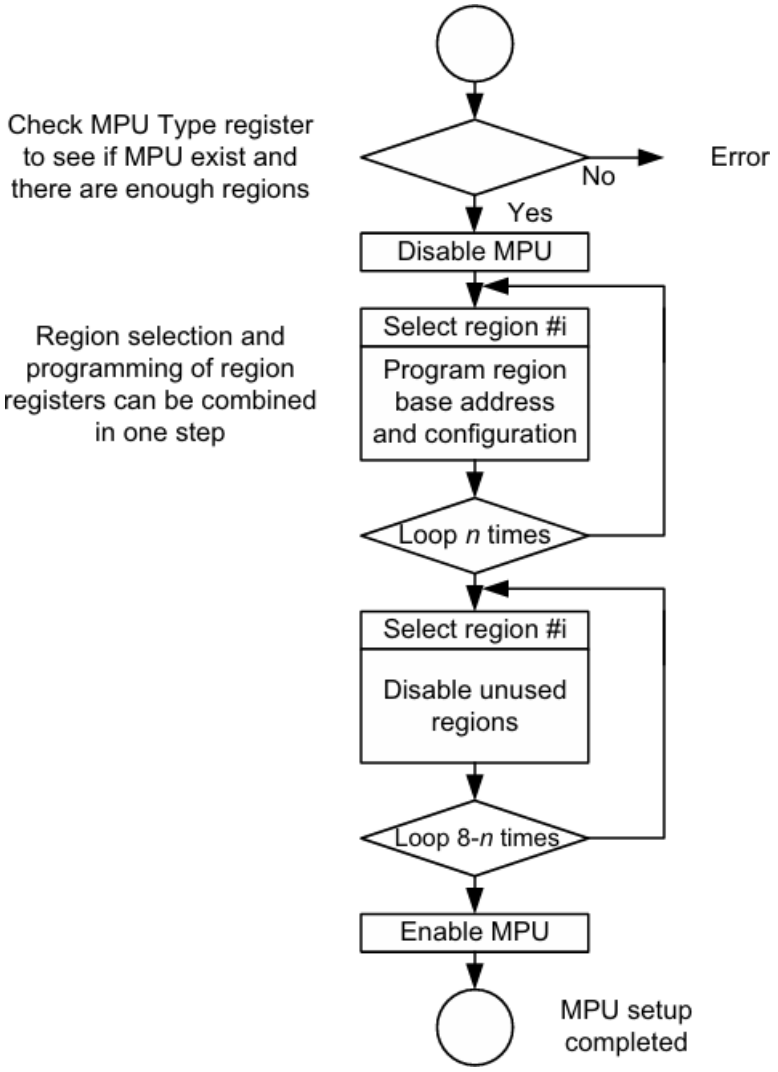
AIM: Write a C program to demonstrate the working of Memory Protection Unit (MPU).

Theory: The Armv8-M architecture supports the optional Protected Memory System Architecture (PMSAv8) as an architecture extension. This extension provides a Memory Protection Unit (MPU) in the Cortex-M processor series. In Arm Cortex-M processors, the number of regions is configurable by silicon designers, and can be up to 16 regions in current Armv8-M Cortex-M processors (Cortex-M33, Cortex-M55, Cortex-M85). The MPU is a programmable device that can define memory access permissions, such as privileged access only, and memory attributes, for example Cache ability, for different memory regions. The MPU can make an embedded system more robust and in some cases it can make the system more secure by:

- Preventing application tasks from corrupting stack or data memory used by other tasks and the OS kernel
- Preventing unprivileged tasks from accessing certain peripherals that can be critical to reliability or the security of the system
- Defining SRAM or RAM space as non-executable (Execute-never, XN) to prevent code injection attacks
- Using the Privileged Execute-never (PXN) feature to prevent accidental execution of user code and data by the OS

The MPU must be programmed by privileged software and should be configured and enabled before use. If the MPU is not enabled, then from a software point of view the MPU is not present. The MPU can also be used to define memory attributes such as Cacheability which can be exported to a system-level cache or to other memory controllers. If a memory accesses violates the access permissions defined by the MPU or accesses a memory location that is not defined by an MPU region, then the transfer would be blocked and a MemManage fault exception would be triggered.

Steps to setup MPU:



MPU Register:

1. **MPU type register:**

   The first one is the MPU Type register. The MPU Type register can be used to deter mine whether the MPU is fitted. If the DREGION field is read as 0, the MPU is not implemented.

2. **MPU control register:**

   The MPU is controlled by a number of registers. The first one is the MPU Control register . This register has three control bits. After reset, the value of this register is zero, which disables the MPU. To enable the MPU, the software should first set up the settings for each MPU region, and then set the ENABLE bit in the MPU Control register.

Table 11.1 Summary of the MPU Registers

Address	Register	CMSIS-Core Symbol	Function
0xE000ED90	MPU Type Register	MPU->TYPE	Provides information about the MPU
0xE000ED94	MPU Control Register	MPU->CTRL	MPU enable/disable and background region control
0xE000ED98	MPU Region Number Register	MPU->RNR	Select which MPU region to be configured
0xE000ED9C	MPU Region Base Address Register	MPU->RBAR	Defines base address of a MPU region
0xE000EDA0	MPU Region Base Attribute and Size Register	MPU->RASR	Defines size and attributes of a MPU region
0xE000EDA4	MPU Alias 1 Region Base Address Register	MPU->RBAR_A1	Alias of MPU->RBAR
0xE000EDA8	MPU Alias 1 Region Base Attribute and Size Register	MPU->RASR_A1	Alias of MPU->RASR
0xE000EDAC	MPU Alias 2 Region Base Address Register	MPU->RBAR_A2	Alias of MPU->RBAR
0xE000EDB0	MPU Alias 2 Region Base Attribute and Size Register	MPU->RASR_A2	Alias of MPU->RASR
0xE000EDB4	MPU Alias 3 Region Base Address Register	MPU->RBAR_A3	Alias of MPU->RBAR
0xE000EDB8	MPU Alias 3 Region Base Attribute and Size Register	MPU->RASR_A3	Alias of MPU->RASR

Table 11.2 MPU Type Register (MPU->TYPE, 0xE000ED90)

Bits	Name	Type	Reset Value	Description
23:16	IREGION	R	0	Number of instruction regions supported by this MPU; because ARMv7-M architecture uses a unified MPU, this is always 0.
15:8	DREGION	R	0 or 8	Number of regions supported by this MPU; in the Cortex®-M3 and Cortex-M4 processors this is either 0 (MPU not present) or 8 (MPU present).
0	SEPARATE	R	0	This is always 0 as the MPU is unified.

Table 11.3 MPU Control Register (MPU->CTRL, 0xE000ED94)

Bits	Name	Type	Reset Value	Description
2	PRIVDEFENA	R/W	0	Privileged default memory map enable. When set to 1 and if the MPU is enabled, the default memory map will be used for privileged accesses as a background region. If this bit is not set, the background region is disabled and any access not covered by any enabled region will cause a fault.
1	HFNMIENA	R/W	0	If set to 1, it enables the MPU during the HardFault handler and NMI handler; otherwise, the MPU is not enabled for the HardFault handler and NMI.
0	ENABLE	R/W	0	Enables the MPU if set to 1.

Configurable Fault Status Register

The CFSR indicates the cause of a MemManage fault, BusFault, or UsageFault



The CFSR is byte accessible. You can access the CFSR or its subregisters as follows:

- access the complete CFSR with a word access to 0xE000ED28
- access the MMFSR with a byte access to 0xE000ED28
- access the MMFSR and BFSR with a halfword access to 0xE000ED28
- access the BFSR with a byte access to 0xE000ED29
- access the UFSR with a halfword access to 0xE000ED2A.

1. **MemManage fault:**

Table 12.3 MemManage Fault Status Register (lowest byte in SCB->CFSR)

Bits	Name	Type	Reset Value	Description
7	MMARVALID	–	0	Indicates the MMFAR is valid
6	–	–	– (read as 0)	Reserved
5	MLSPERR	R/Wc	0	Floating point lazy stacking error (available on Cortex®-M4 with floating point unit only)
4	MSTKERR	R/Wc	0	Stacking error
3	MUNSTKERR	R/Wc	0	Unstacking error
2	–	–	– (read as 0)	Reserved
1	DACCVIOL	R/Wc	0	Data access violation
0	IACCVIOL	R/Wc	0	Instruction access violation

2. **Bus Fault Status Register**

Table 12.4 Bus Fault Status Register (2nd byte in SCB->CFSR)

CFSR Bits	Name	Type	Reset Value	Description
15	BFARVALID	–	0	Indicates BFAR is valid
14	–	–	–	–
13	LSPERR	R/Wc	0	Floating point lazy stacking error (available on Cortex®-M4 with floating point unit only)
12	STKERR	R/Wc	0	Stacking error
11	UNSTKERR	R/Wc	0	Unstacking error
10	IMPRECISERR	R/Wc	0	Imprecise data access error
9	PRECISERR	R/Wc	0	Precise data access error
8	IBUSERR	R/Wc	0	Instruction access error

3. **Usage Fault Register**

Table 12.5 Usage Fault Status Register (Upper half-word in SCB->CFSR)

CFSR Bits	Name	Type	Reset Value	Description
25	DIVBYZERO	R/Wc	0	Indicates a divide by zero has taken place (can be set only if DIV_0_TRP is set)
24	UNALIGNED	R/Wc	0	Indicates that an unaligned access fault has taken place
23:20	–	–	–	–
19	NOCP	R/Wc	0	Attempts to execute a coprocessor instruction
18	INVPC	R/Wc	0	Attempts to do an exception with a bad value in the EXC_RETURN number
17	INVSTATE	R/Wc	0	Attempts to switch to an invalid state (e.g., ARM)
16	UNDEFINSTR	R/Wc	0	Attempts to execute an undefined instruction

Program

```
#include "LPC17xx.h"

#define PROT_BASE 0x2007E000UL  // AHB SRAM, normally unused by default projects

void MemManage_Handler(void) {
   volatile uint32_t cfsr  = SCB->CFSR;  // Expect 0x00000082
   volatile uint32_t mmfar = SCB->MMFAR;  // Expect 0x2007E000
   while (1); // halt for debug
}

int main(void) {
   // 1) Enable MemManage faults so they don't escalate to HardFault
   SCB->SHCSR |= SCB_SHCSR_MEMFAULTENA_Msk;

   // (Optional) clear any stale flags
   SCB->CFSR = 0xFFFFFFFFUL; (void)SCB->CFSR;

   // 2) Configure MPU Region 0: 256B at 0x2007E000 as NO ACCESS
   MPU->RNR  = 0;
   MPU->RBAR = PROT_BASE;                    // base must be aligned to region size
   MPU->RASR = ((8 - 1) << MPU_RASR_SIZE_Pos) | // SIZE=8 ? 2^(8)=256 B
           (0U << MPU_RASR_AP_Pos)     | // AP=0b000 ? No access (priv & unpriv)
           MPU_RASR_ENABLE_Msk;

   // 3) Enable MPU; keep PRIVDEFENA so normal regions still work
   MPU->CTRL = MPU_CTRL_ENABLE_Msk | MPU_CTRL_PRIVDEFENA_Msk;

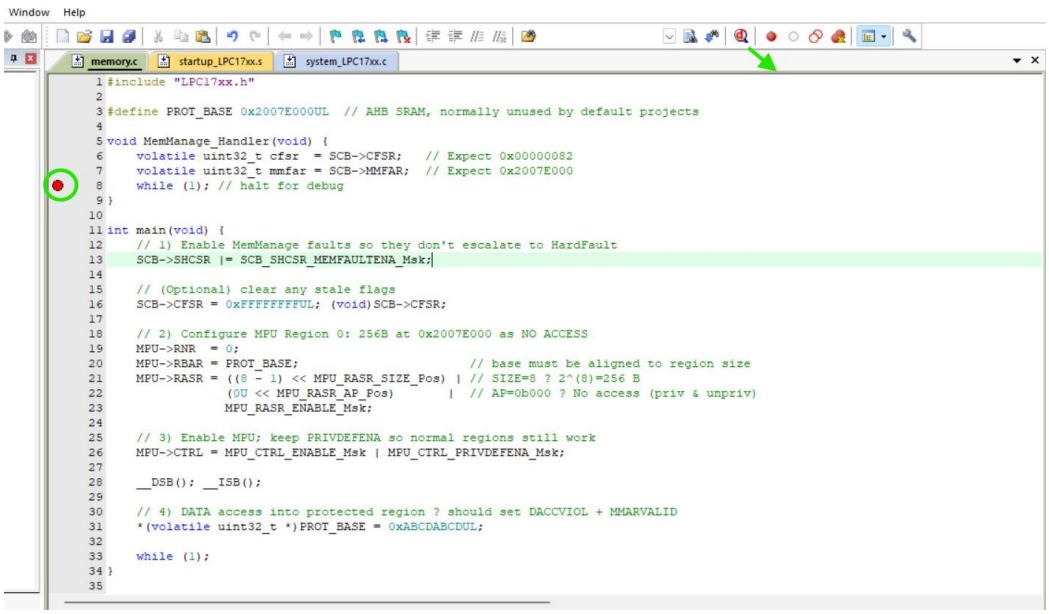
   __DSB(); __ISB();

   // 4) DATA access into protected region ? should set DACCVIOL + MMARVALID
   *(volatile uint32_t *)PROT_BASE = 0xABCDABCDUL;

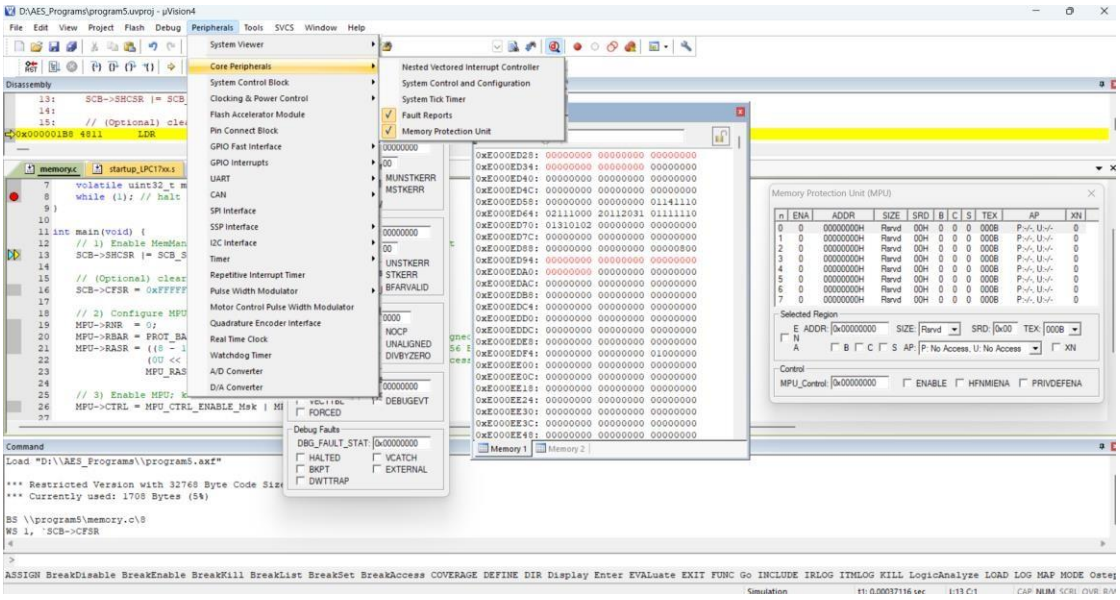
   while (1);
}
```

Steps to execute the program:

1. All Steps for embedded C program remain the same.
2. Set up breakpoint in line number 8:



3. After Debug → Goto Peripherals→ Open → **Core Peripherals → Fault Reports & Memory Protection Unit**.

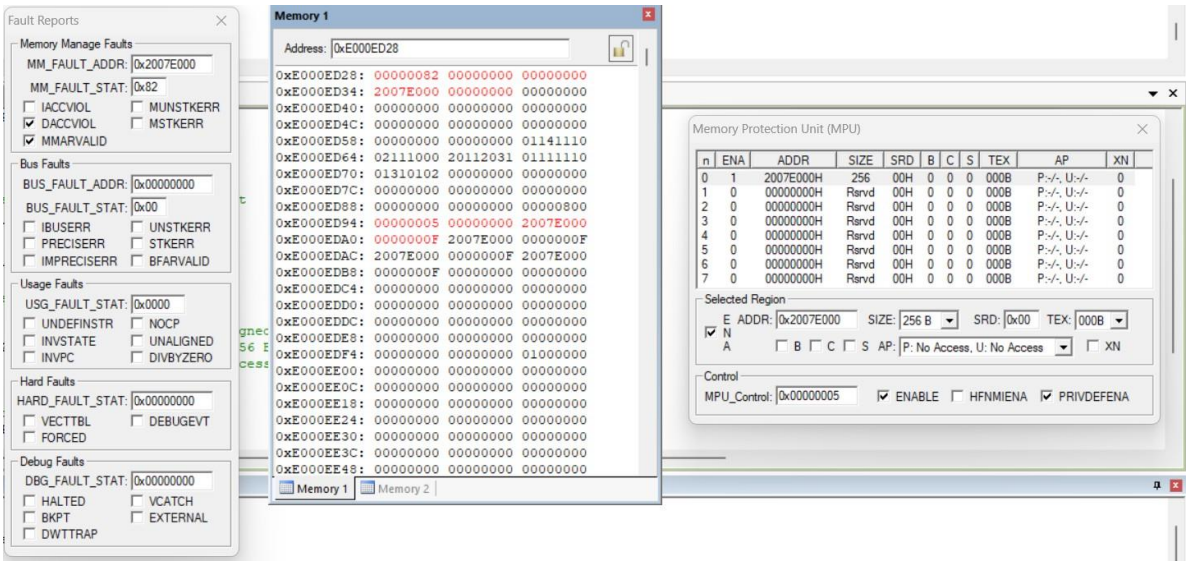


4. Run the program –F5

   SCB->CFSR = 0x00000082

- In binary: 0000 0000 1000 0010
- Bits set: **bit 1** (DACCVIOL) + **bit 7** (MMARVALID)
- data access violation occurred, and the fault address in SCB->MMFAR is valid.

- Address in memory 0xE000ED28 has 0x00000082 implying data violation

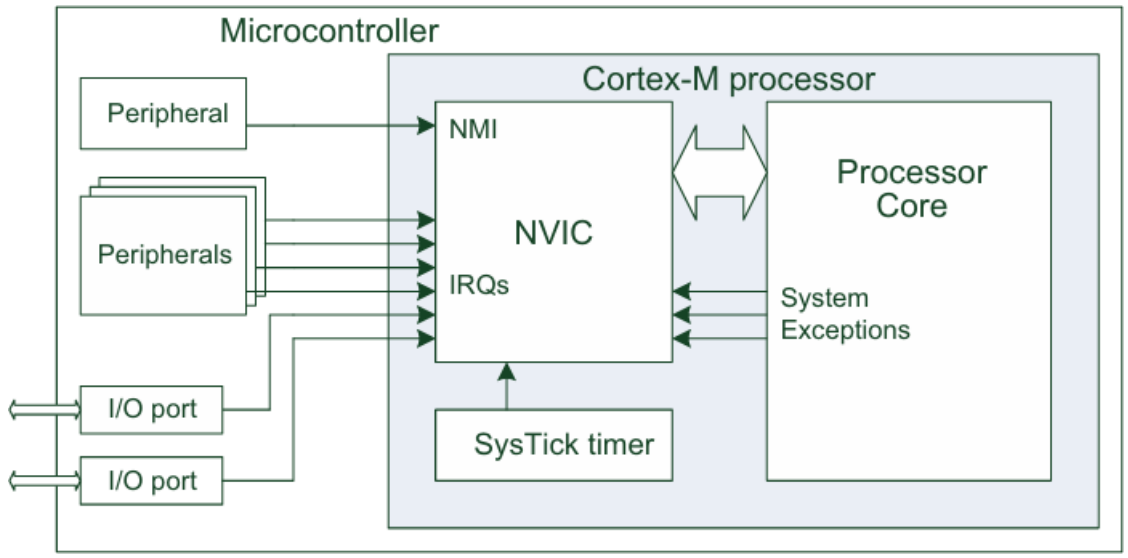


Experiment 5

AIM: **Write a C program to demonstrate the use of the SysTick timer interrupt managed by Nested Vector Interrupt Controller(NVIC).**

Theory: All Cortex-M processors include a Nested Vectored Interrupt Controller (NVIC) to manage interrupts in an efficient and prioritized way. In ARM terminology, an interrupt is just one type of exception, and exceptions are events that cause the processor to temporarily halt normal execution and run a special routine called an exception handler. Besides peripheral interrupts, Cortex-M processors also support fault exceptions such as HardFault, BusFault, UsageFault, and MemManage fault, which are triggered when errors occur during program execution. They also include system exceptions to support operating system features, such as the SysTick timer interrupt for periodic tasks, PendSV for context switching, and the SVC (Supervisor Call) instruction for invoking OS services.

In a typical Cortex-M microcontroller ,the NVIC receives interrupt requests from various sources



The Cortex-M processors provide a feature-packed exception architecture that sup ports a number of system exceptions and external interrupts. Exceptions are numbered 1e15 for system exceptions and 16 and above for interrupt inputs (inputs to the processor, but not necessarily accessible on the I/O pins of the package). Most of the exceptions, including all interrupts, have programmable priorities, and a few system exceptions have fixed priority. Different Cortex-M3 or Cortex-M4 microcontrollers can have different numbers of interrupt sources (from 1e240) and different numbers of priority levels. This is because chip designers can configure the Cortex-M3 or Cortex-M4 design source code for different application requirements.

Table 7.1 List of System Exceptions

Exception Number	Exception Type	Priority	Descriptions
1	Reset	−3 (Highest)	Reset
2	NMI	−2	Non-Maskable Interrupt (NMI), can be generated from on chip peripherals or from external sources.
3	Hard Fault	−1	All fault conditions, if the corresponding fault handler is not enabled
4	MemManage Fault	Programmable	Memory management fault; MPU violation or program execution from address locations with XN (eXecute Never) memory attribute.
5	Bus Fault	Programmable	Bus error; usually occurs when AHB interface receives an error response from a bus slave (also called *prefetch abort* if it is an instruction fetch or *data abort* if it is a data access). Can also be caused by other illegal accesses.
6	Usage Fault	Programmable	Exceptions due to program error or trying to access co-processor (the Cortex-M3 and Cortex-M4 processor do not support co-processors).
7–10	Reserved	NA	–
11	SVC	Programmable	SuperVisor Call; usually used in OS environment to allow application tasks to access system services.
12	Debug Monitor	Programmable	Debug monitor; exception for debug events like breakpoints, watchpoints when software based debug solution is used.
13	Reserved	NA	–
14	PendSV	Programmable	Pendable service call; An exception usually used by an OS in processes like context switching.
15	SYSTICK	Programmable	System Tick Timer; Exception generates by a timer peripheral which is included in the processor. This can be used by an OS or can be used as a simple timer peripheral.

SYSTEM TICK TIMER(SysTick):

The SysTick timer contains four registers. A data structure called SysTick is defined in the CMSIS-Core header file to allow these registers to be accessed easily. The counter inside the SysTick is a 24-bit decrement counter. It can decrement using the processor's clock, or using a reference clock signal (called STCLK in Cortex-M3 or Cortex-M4 Technical Reference Manual from ARM). The exact implementation of the reference clock signal depends on the microcontroller design. In some cases it might not be available. The reference clock must be at least two times slower than the processor clock because of the sampling logic used for rising edge detection. When the counter is enabled by setting bit 0 of the Control and Status register,

the current value register decrements at every processor clock cycle or every rising edge of the reference clock. If it reaches zero, it will then load the value from the reload value register and continue.

Details of the SysTick registers:

Table 9.6 Summary of the SysTick Registers

Address	CMSIS-Core Symbol	Register
0xE000E010	SysTick->CTRL	SysTick Control and Status Register
0xE000E014	SysTick->LOAD	SysTick Reload Value Register
0xE000E018	SysTick->VAL	SysTick Current Value Register
0xE000E01C	SysTick->CALIB	SysTick Calibration Register

Table 9.7 SYSTICK Control and Status Register (0xE000E010)

Bits	Name	Type	Reset Value	Description
16	COUNTFLAG	R	0	Read as 1 if counter reaches 0 since last time this register is read; clear to 0 automatically when read or when current counter value is cleared
2	CLKSOURCE	R/W	0	0 = External reference clock (STCLK) 1 = Use core clock
1	TICKINT	R/W	0	1 = Enable SYSTICK interrupt generation when SYSTICK timer reaches 0 0 = Do not generate interrupt
0	ENABLE	R/W	0	SYSTICK timer enable

Table 9.8 SYSTICK Reload Value Register (0xE000E014)

Bits	Name	Type	Reset Value	Description
23:0	RELOAD	R/W	0	Reload value when timer reaches 0

Table 9.9 SYSTICK Current Value Register (0xE000E018)

Bits	Name	Type	Reset Value	Description
23:0	CURRENT	R/Wc	0	Read to return current value of the timer. Write to clear counter to 0. Clearing of current value also clears COUNTFLAG in SYSTICK Control and Status Register

Steps to configure systick timer:

1. Disable the SysTick timer by writing 0 to SysTick->CTRL. This step is optional. It is recommended for reusable code because the SysTick could have been enabled previously.
2. Write the new reload value to SysTick->LOAD. The reload value should be the interval value −1.
3. Write to the SysTick Current Value register SysTick->VAL with any value to clear the current value to 0.
4. Write to the SysTick Control and Status register SysTick->CTRL to start the SysTick timer.

PROGRAM:

```
#include "LPC17xx.h"

void SysTick_Handler(void);  // SysTick ISR


void init_GPIO(void) {
   // Configure P0.22 as output (LED)
   LPC_PINCON->PINSEL1 &= ~(3 << 12);   // P0.22 = GPIO
   LPC_GPIO0->FIODIR   |=  (1 << 22);   // Set as output
}

void init_SysTick(void) {
   // Configure SysTick to trigger every 0.5s (assuming 100 MHz clock)
   SysTick->LOAD = (SystemCoreClock / 2) - 1; // Reload value
   SysTick->VAL  = 0;                         // Clear current value
   SysTick->CTRL = SysTick_CTRL_TICKINT_Msk |  // Enable interrupt
                   SysTick_CTRL_ENABLE_Msk;   // Enable SysTick
   // No need to enable in NVIC manually, SysTick is part of Cortex core
}

// SysTick interrupt handler
void SysTick_Handler(void) {
   // Toggle LED on P0.22
   LPC_GPIO0->FIOPIN ^= (1 << 22);
}

int main(void) {
   SystemInit();     // Setup system clock
   SystemCoreClockUpdate();

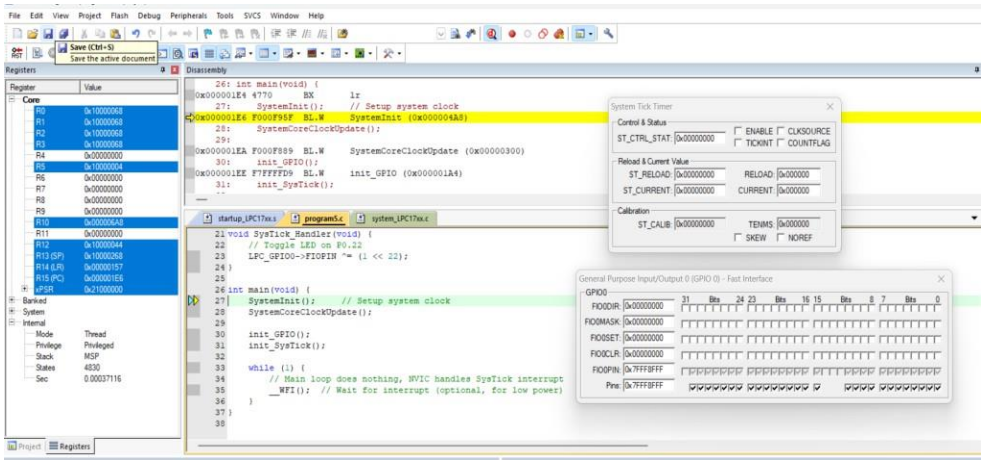
   init_GPIO();
   init_SysTick();

   while (1) {
```

```
    // Main loop does nothing, NVIC handles SysTick interrupt
    __WFI();  // Wait for interrupt (optional, for low power)
  }
}
```

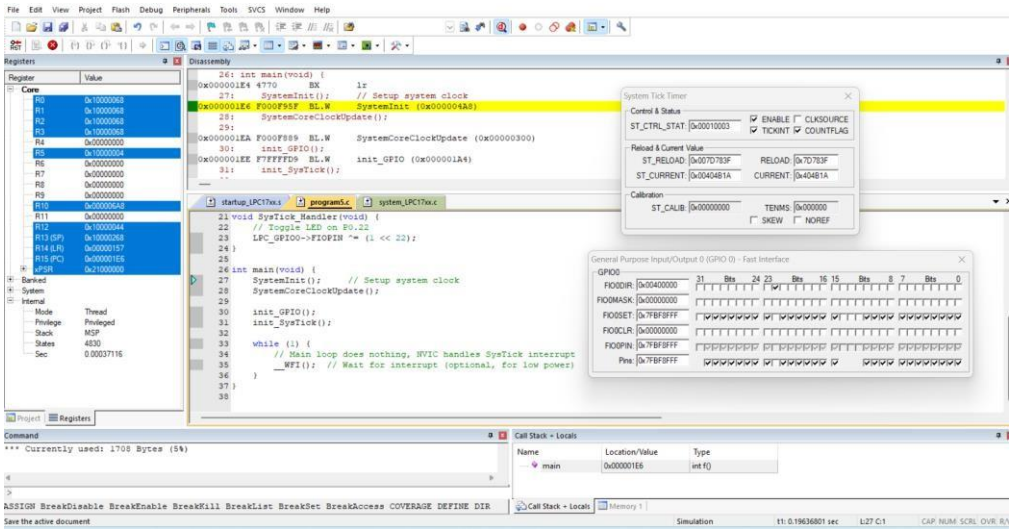
Steps to execute the program:

1. All Steps for embedded C program remain the same.
2. After Debug goto → Peripherals → Fast Gpio port0 →Peripherals → System Tick Timer(to view systick timer registers)



Output:

The SysTick timer generates an interrupt every **0.5 seconds**. Each time the interrupt occurs, the LED connected to **P0.22** toggles its state.



Time period calculation:

Assume Clock frequency = 100MHz=fclk, T = 0.5s

Load = (fclk * T)-1 = 50Mhz-1

Indicated in line: SysTick->LOAD = (SystemCoreClock / 2) - 1

Experiment 76

AIM: **Write an embedded C program to generate a sine wave using SysTick timer in LPC1768.**

Theory: The following program generates a sine wave using the DAC output pin P0.26(AOUT) of LPC1768 microcontroller. A sine lookup table of 100 samples is created, and the SysTick timer interrupt is used to periodically send these values to the DAC.

Program:

```
#include <LPC17xx.h>
#include <math.h>

#define PI 3.14159265
#define SAMPLES 100  // number of points per cycle

uint16_t sine_table[SAMPLES];

// ---------- Generate Sine Table ----------
void generate_sine_table(void) {
   int i;  // declare outside
   for (i = 0; i < SAMPLES; i++) {
      // Sine wave between 0 and 1023
      sine_table[i] = (uint16_t)(512 + 511 * sin(2 * PI * i / SAMPLES));
   }
}

// ---------- Initialize DAC (P0.26) ----------
void init_DAC(void) {
   LPC_PINCON->PINSEL1 &= ~(3 << 20);  // Clear bits
   LPC_PINCON->PINSEL1 |=  (2 << 20);  // P0.26 -> AOUT (DAC)
}

// ---------- Send value to DAC ----------
void DAC_Output(uint16_t value) {
   LPC_DAC->DACR = (value << 6);  // DACR[15:6] = 10-bit value
}

// ---------- SysTick ISR ----------
volatile uint32_t index = 0;
void SysTick_Handler(void) {
   DAC_Output(sine_table[index++]);
```

```
    if (index >= SAMPLES) index = 0;
}

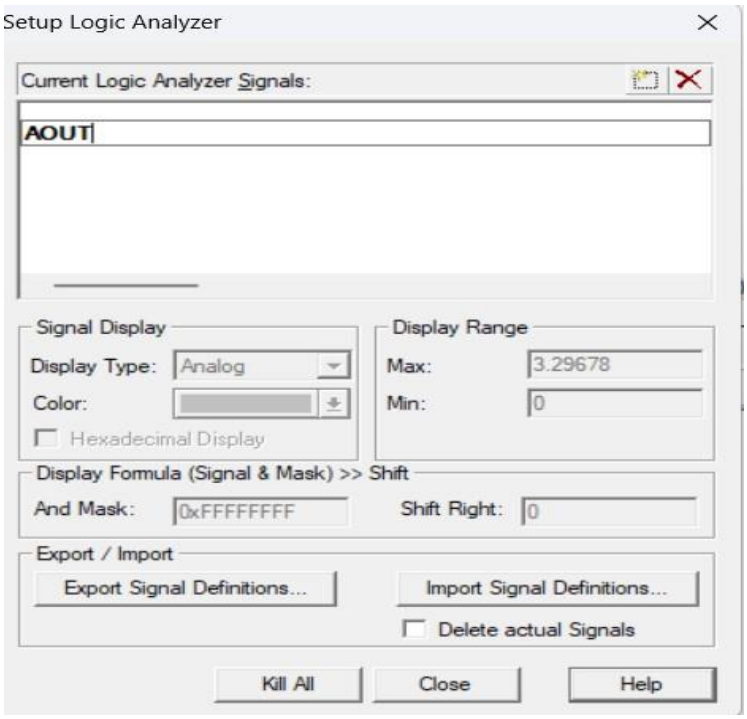

// ---------- Main ----------
int main(void) {
    generate_sine_table();
    init_DAC();

    // SysTick: interrupt at sample rate
    // Example: For 1 kHz sine wave:
    // Fs = 100kHz, Samples = 100
    // SysTick = SystemCoreClock / Fs = 100 MHz / 100k = 1000
    SysTick_Config(SystemCoreClock / (1000 * SAMPLES));

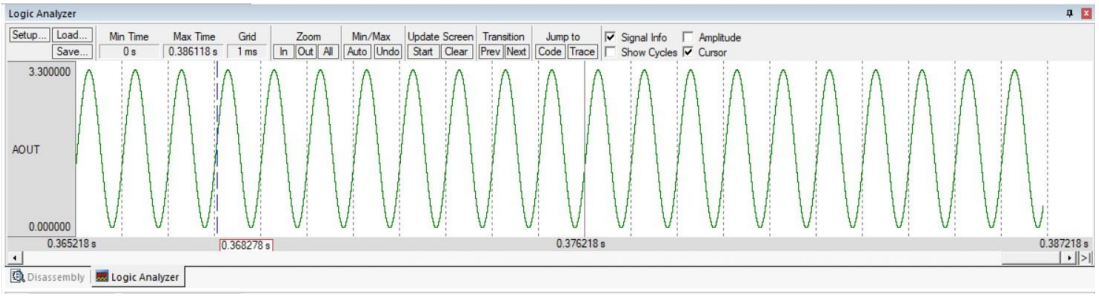
    while (1) {
        // CPU free, ISR handles waveform
    }
}
```

Steps to execute:

1. All C program steps remain the same.
2. After Debug goto → View → Analysis window → Logic Analyser → Setup → Insert New signal → Type "AOUT"→ Press enter.



3. After inserting signal click on run (F5): Output



Calculations:

- fsine = 1KHz, N = 100 samples/cycle(number of points per cycle)
- Sampling frequency = fs = fsine * N
- As per data = > fs = 1000 * 100 = 1,00,000→ SysTick must trigger **100,000 times per second**.
- Indicated in this line → SysTick_Config(SystemCoreClock / (1000 * SAMPLES));
- ISR (Interrupt service routine) outputs 100 samples per cycle.
- fout=fs/N = 100000/100 = 1000=1KHz.
- LPC1768 has an inbuilt DAC with Vref tied to 3.3V. Hence the output of sine wave is from 0 to 3.3V.

Experiment 7

AIM: **Write a C program for ARM Cortex-M3 (LPC1768) to demonstrate the working of low power modes: Idle, Sleep, Deep Sleep, and Power Down.**

Theory: In this program, P0.22, P0.23, P0.24, and P0.25 are configured as output pins to drive LEDs.
Each time the LPC1768 wakes up from a low power mode (Idle, Sleep, Deep Sleep, or Power Down), the corresponding LED pin is toggled to indicate the mode from which the processor has resumed operation.
The processor is placed into the selected low power mode by writing the appropriate value to the PCON register, and it is brought out of that mode by an external interrupt (EINT0) configured on pin P2.10.

- **System Control Register:**
  The Cortex-M processor provides a register called the System Control Register (SCR) to allow you to select between the Sleep mode and Deep Sleep mode. This register is at address 0xE000ED10, and can be accessed in C programming using the "SCB->SCR" symbol. The SLEEPDEEP bit (bit2) can be set to enable the DeepSleepmode. This register can also be used to control other lower power features like Sleep-On-Exit and SEV-On Pend.

Table 9.2 System Control Register (SCB->SCR, 0xE000ED10)

Bits	Name	Type	Reset Value	Description
4	SEVONPEND	R/W	0	Send Event on Pending; when this is set to 1, the processor wakes up from WFE if a new interrupt is pended, regardless of whether the interrupt has priority higher than the current level and whether it was enabled
3	Reserved	–	–	–
2	SLEEPDEEP	R/W	0	When set to 1, the Deep Sleep mode is selected. Otherwise the sleep mode is selected.
1	SLEEPONEXIT	R/W	0	When this bit is set to 1, it enables the Sleep-On-Exit feature, which cause the processor to enter sleep mode automatically when exiting an exception handler and is returning to Thread.
0	Reserved	–	–	–

- The processor provides two instructions for entering sleepmodes: WFI and WFE

Table 9.3 Instructions for Entering Sleep Mode

Instruction	CMSIS-Core	Descriptions
WFI	void __WFI(void);	Wait for Interrupt Enter sleep mode. The processor can wake-up by interrupt request, debug request or reset.
WFE	void __WFE(void);	Wait for Event Enter sleep mode conditionally. If the internal event register is clear, the processor enters sleep mode. Otherwise the internal event register is cleared and the processor continues. The processor can wake-up by interrupt request, event input, debug request or reset.

- PCON (Power Control Register) is used to control power-down modes in LPC1768.

PCON Value	Mode Selected	Description
0x01	Idle Mode	CPU clock stopped. Peripherals, system clock, and PLL remain active. MCU wakes up on any interrupt.
0x02	Sleep Mode	Like Idle, but main oscillator can be stopped. Reduces more power. Wakes up on interrupt.
0x03	Deep Sleep	Most clocks off, only watchdog and RTC (if enabled) can run. SRAM/flash in low-power. Wake-up via interrupt.
0x07	Power Down	Main oscillator, PLL, and most internal circuits powered down. RTC/wakeup logic active. Minimum power mode with RAM retention.

Program:

```
#include <LPC17xx.h>

void EINT0_IRQHandler(void);
void toggleLED(uint32_t pin);

static uint32_t led_state = 0;  // Track LED states in software

void init_GPIO(void) {
   // Configure P0.22, P0.23, P0.24, P0.25 as outputs
   LPC_PINCON->PINSEL1 &= ~((3 << 12) | (3 << 14) | (3 << 16) | (3 << 18));
   LPC_GPIO0->FIODIR  |=  (0x0F << 22);  // Set P0.22–P0.25 as outputs
}

void init_EINT0(void) {
   // Configure P2.10 as EINT0
   LPC_PINCON->PINSEL4 &= ~(3 << 20);
   LPC_PINCON->PINSEL4 |=  (1 << 20);

   // Falling edge sensitive
   LPC_SC->EXTMODE  |= (1 << 0);
```

```
    LPC_SC->EXTPOLAR &= ~(1 << 0);

    NVIC_EnableIRQ(EINT0_IRQn);
}

int main(void) {
    SystemInit();
    init_GPIO();
    init_EINT0();

    while (1) {
        // -------- IDLE MODE --------
        LPC_SC->PCON = 0x01;
        __WFI();
        toggleLED(22);

        // -------- SLEEP MODE --------
        LPC_SC->PCON = 0x02;
        __WFI();
        toggleLED(23);

        // -------- DEEP SLEEP MODE --------
        LPC_SC->PCON = 0x03;
        __WFI();
        toggleLED(24);

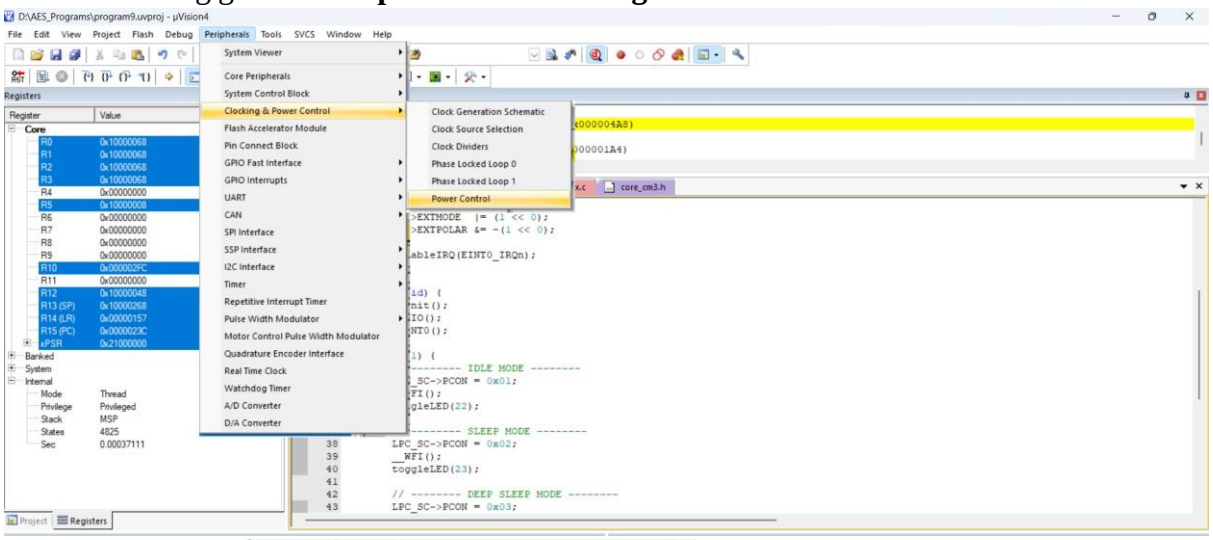
        // -------- POWER DOWN MODE --------
        LPC_SC->PCON = 0x07;
        __WFI();
        toggleLED(25);
    }
}

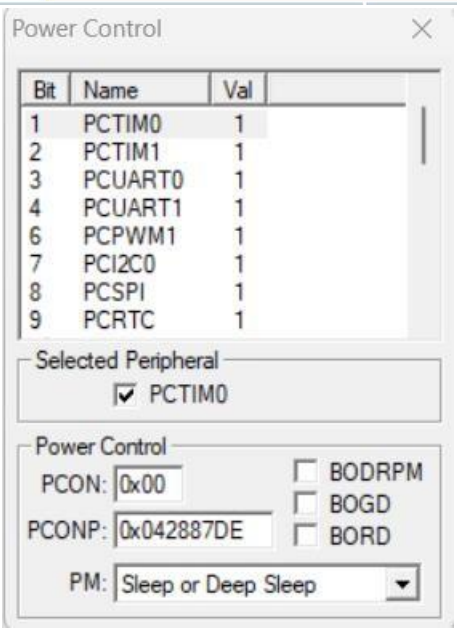
void EINT0_IRQHandler(void) {
    LPC_SC->EXTINT = (1 << 0);   // Clear EINT0 interrupt flag
}

void toggleLED(uint32_t pin) {
    led_state ^= (1 << pin);     // Flip software state
    if (led_state & (1 << pin)) {
        LPC_GPIO0->FIOSET = (1 << pin);   // Turn ON LED
    } else {
        LPC_GPIO0->FIOCLR = (1 << pin);   // Turn OFF LED
    }
}
```

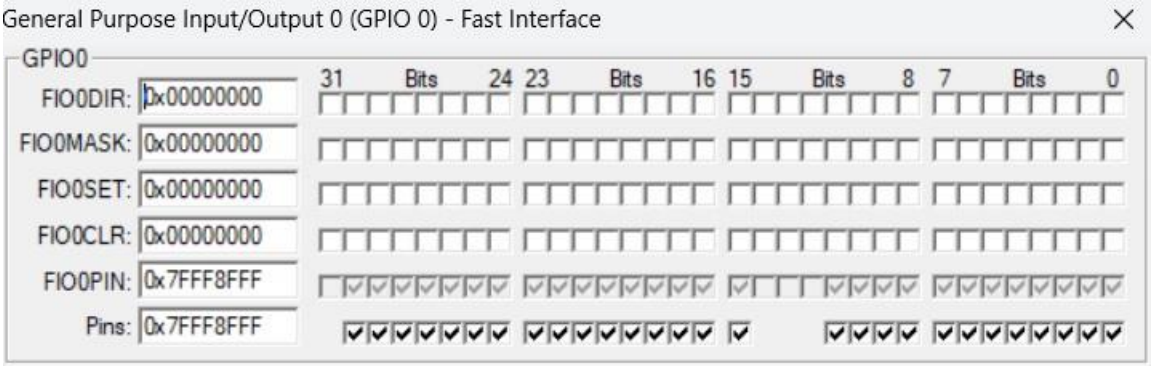
Steps to execute:

1. All C program steps remain the same.
2. After Debug goto → **Peripherals → Clocking and Power Control → Power Control**

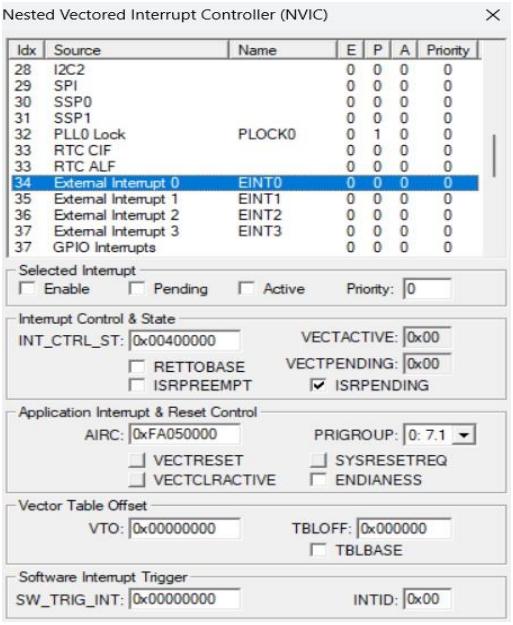




3. Open GPIO Fast Interface → GPIO PORT0



4. Goto → Peripherals → Core peripherals → Nested Vector Interrupt Controller → select External Interrupt 0(EINT0).

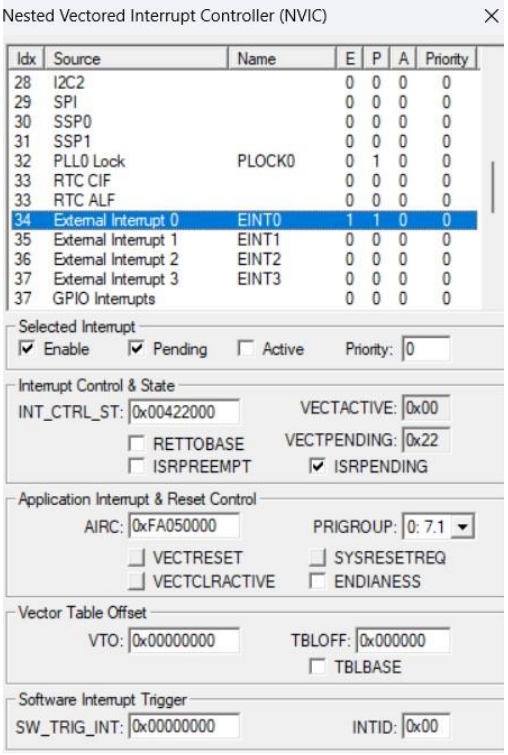


5. Program flow:

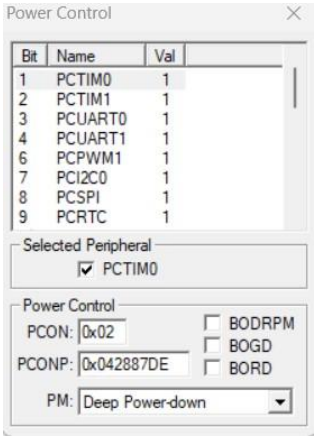
Low Power Mode Entered	PCON Value	Wake-up Source	LED Pin Toggled	Observation in Simulator
Idle Mode	0x01	EINT0 (P2.10)	P0.22	P0.22 toggles ON/OFF
Sleep Mode	0x02	EINT0 (P2.10)	P0.23	P0.23 toggles ON/OFF
Deep Sleep Mode	0x03	EINT0 (P2.10)	P0.24	P0.24 toggles ON/OFF
Power Down Mode	0x07	EINT0 (P2.10)	P0.25	P0.25 toggles ON/OFF

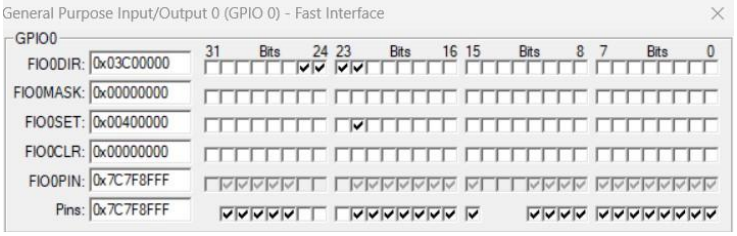
6. Run the program continuously by clicking F5.

7. Stop the execution. Check the line of execution, enable the interrupt by selecting EINT0 and click on Pending in NVIC table. Then run the program.

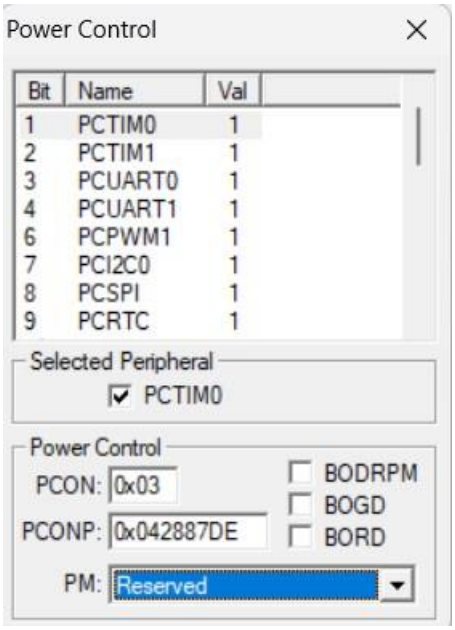


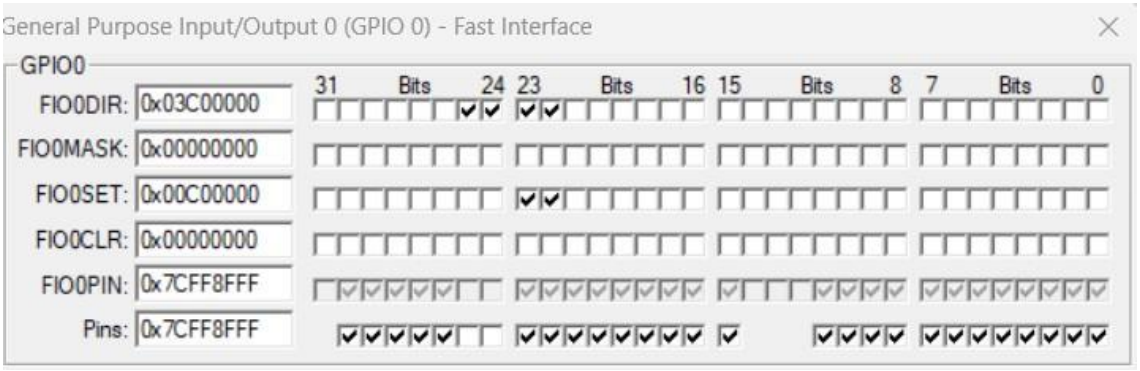
8. The processor goes from IDLE to Sleep mode indicated as PCON -0X02 in power control. Also P0.22 goes from 0-1(toggle).



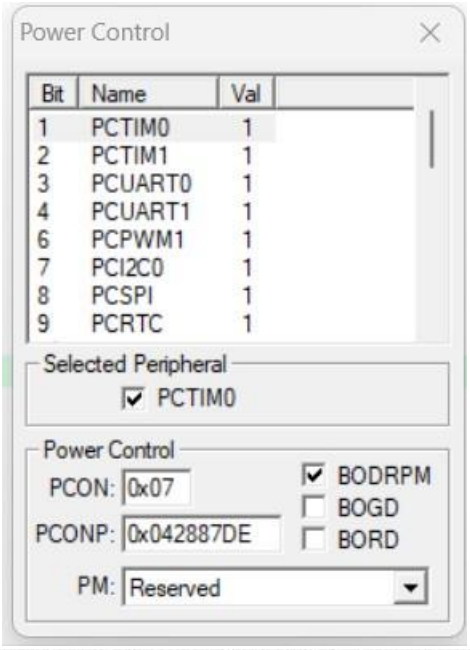


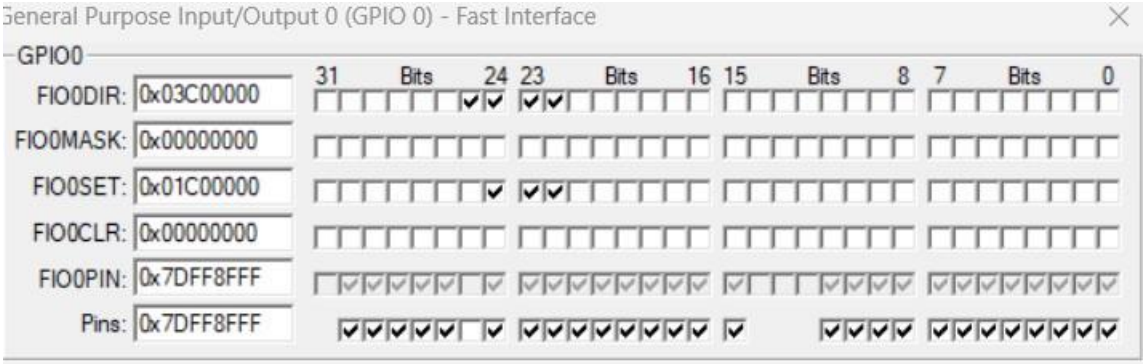
9. Repeat the procedure by enabling interrupt.
   The processor goes from SLEEP Mode to DEEP SLEEP mode. Indicated as PCON→ 0X03. P0.23 goes from 0 –1 (toggle).



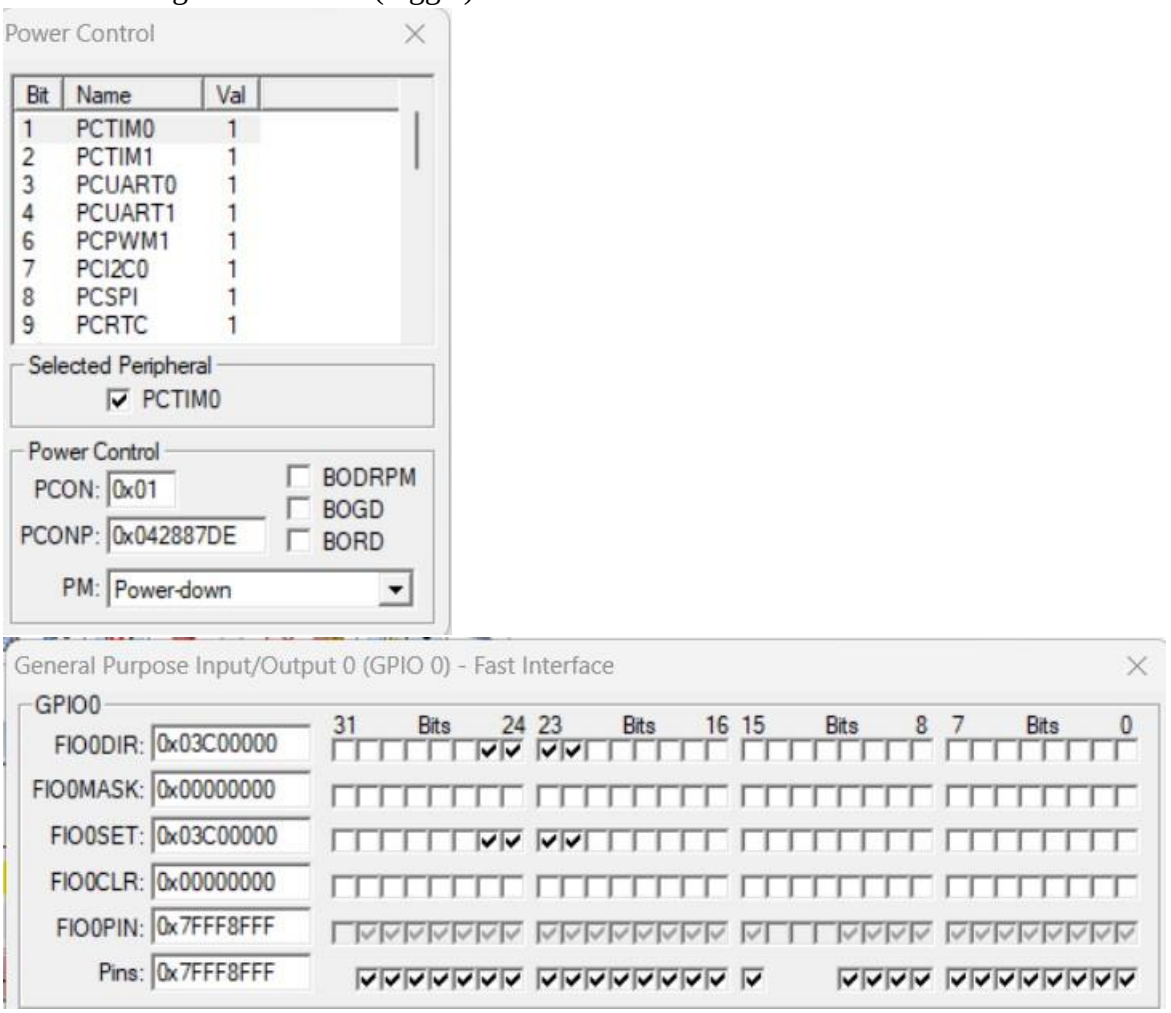


10. Repeat the procedure by enabling interrupt.
    The processor goes from DEEP SLEEP to POWER DOWN mode. Indicated as PCON→ 0X07. P0.24 goes from 0 –1 (toggle).





11. Repeat the procedure by enabling interrupt.
The processor goes back from POWER DOWN to IDLE mode. Indicated as PCON→ 0X01. P0.25 goes from 0 –1 (toggle).

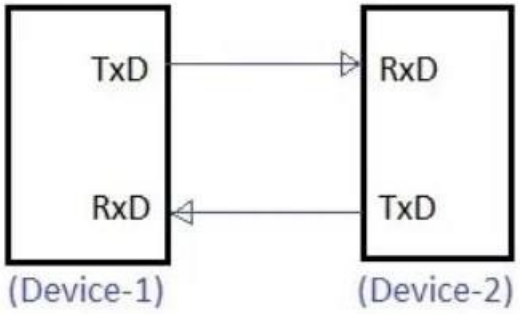


Note: Each time the interrupt must be enabled for processor to go from one low power state to other.

Experiment 8

AIM: **Write an ARM C program for the LPC1768 microcontroller to initialize UART0 for serial communication at 9600 baud rate and send a string repeatedly in an infinite loop with a simple software delay between transmissions.**

Theory: A Universal Asynchronous Receiver/Transmitter (UART) is a hardware communication block that consists of a transmitter and a receiver, which communicate with each other using a simple two-wire interface. A shared baud rate generator ensures that both sides are configured to transmit and receive data at the same speed.



LPC1768 has **4 UART interfaces**:

- UART0
- UART1
- UART2
- UART3

This program uses UART0. The registers used by UART module is given below:

Register	Description
RBR	Contains the recently received Data
THR	Contains the data to be transmitted
FCR	FIFO Control Register
LCR	Controls the UART frame formatting (Number of Data Bits, Stop bits, Parity)
DLL	Least Significant Byte of the UART baud rate generator value
DLM	Most Significant Byte of the UART baud rate generator value

1. FCR (FIFO Control Register)

LPC1768 has inbuilt 16byte FIFO for Receiver/Transmitter. Thus it can store 16-bytes of data received on UART without overwriting. If the data is not read before the Queue (FIFO) is filled then the new data will be lost and the OVERRUN error bit will be set.

FCR						
31:8	7:6	5:4	3	2	1	0
RESERVED	RX TRIGGER	RESERVED	DMA MODE	TX FIFO RESET	RX FIFO RESET	FIFO ENABLE

Bit 0 – FIFO:
This bit is used to enable/disable the FIFO for the data received/transmitted.
0--FIFO is Disabled.
1--FIFO is Enabled for both Rx and Tx.

Bit 1 – RX_FIFO:
This is used to clear the 16-byte Rx FIFO.
0--No impact.
1--CLears the 16-byte Rx FIFO and the resets the FIFO pointer.

Bit 2 – Tx_FIFO:
This is used to clear the 16-byte Tx FIFO.
0--No impact.
1--Clears the 16-byte Tx FIFO and the resets the FIFO pointer.

Bit 3 – DMA_MODE:
This is used for Enabling/Disabling DMA mode.
0--Disables the DMA.
1--Enables DMA only when the FIFO(bit-0) bit is SET.

Bit 7:6 – Rx_TRIGGER:
This bit is used to select the number of bytes of the receiver data to be written so as to enable the interrupt/DMA.
00-- Trigger level 0 (1 character or 0x01)
01-- Trigger level 1 (4 characters or 0x04)
10-- Trigger level 2 (8 characters or 0x08)
11-- Trigger level 3 (14 characters or 0x0E)

2. LCR (Line Control Register)

This register is used for defining the UART frame format ie. Number of Data bits, STOP bits etc.

LCR						
31:8	7	6	5:4	3	2	1:0
Reserved	DLAB	Break COntrol	Parity Select	Parity Enable	Stop Bit Select	Word Length Select

Bit 1:0 – WLS : WordLenghtSelect
These two bits are used to select the character length

00-- 5-bit character length
01-- 6-bit character length
10-- 7-bit character length
11-- 8-bit character length

Bit 2 – Stop Bit Selection:
This bit is used to select the number(1/2) of stop bits
0-- 1 Stop bit
1-- 2 Stop Bits

Bit 3 – Parity Enable:
This bit is used to Enable or Disable the Parity generation and checking.
0-- Disable parity generation and checking.
1-- Enable parity generation and checking.

Bit 5:4 – Parity Selection:
These two bits will be used to select the type of parity.
00-- Odd parity. Number of 1s in the transmitted character and the attached parity bit will be odd.
01-- Even Parity. Number of 1s in the transmitted character and the attached parity bit will be even.
10-- Forced "1" stick parity.
11-- Forced "0" stick parity

Bit 6 – Break Control
0-- Disable break transmission.
1-- Enable break transmission. Output pin UARTn TXD is forced to logic 0

Bit 8 – DLAB: Divisor Latch Access Bit
This bit is used to enable the access to divisor latch.
0-- Disable access to divisor latch
1-- Enable access to divisor latch

3. **LSR (Line Status Register)**
   The is a read-only register that provides status information of the UART TX and RX blocks.

LSR								
31:8	7	6	5	4	3	2	1	0
Reserved	RXFE	TEMT	THRE	BI	FE	PE	OE	RDR

Bit 0 – RDR: Receive Data Ready
This bit will be set when there is a received data in RBR register. This bit will be automatically cleared when RBR is empty.
0-- The UARTn receiver FIFO is empty.
1-- The UARTn receiver FIFO is not empty.

Bit 1 – OE: Overrun Error
The overrun error condition is set when the UART Rx FIFO is full and a new character is received. In this case, the UARTn RBR FIFO will not be overwritten and the character in the UARTn RSR will be lost.

0-- No overrun
1-- Buffer over run

Bit 2 – PE: Parity Error
This bit is set when the receiver detects a error in the Parity.
0-- No Parity Error
1-- Parity Error

Bit 3 – FE: Framing Error
This bit is set when there is error in the STOP bit(LOGIC 0)
0-- No Framing Error
1-- Framing Error

Bit 4 – BI: Break Interrupt
This bit is set when the RXDn is held in the spacing state (all zeroes) for one full character transmission
0-- No Break interrupt
1-- Break Interrupt detected.

Bit 5 – THRE: Transmitter Holding Register Empty
THRE is set immediately upon detection of an empty THR. It is automatically cleared when the THR is written.
0-- THR register is Empty
1-- THR has valid data to be transmitted

Bit 6 – TEMT: Transmitter Empty
TEMT is set when both UnTHR and UnTSR are empty; TEMT is cleared when any of them contain valid data.
0-- THR and/or the TSR contains valid data.
1-- THR and the TSR are empty.

Bit 7 – RXFE: Error in Rx FIFO
This bit is set when the received data is affected by Framing Error/Parity Error/Break Error.
0-- RBR contains no UARTn RX errors.
1-- RBR contains at least one RX error.

4. **TER (Transmitter Enable register)**
   This register is used to Enable/Disable the transmission

TER		
31:8	7	6-0
Reserved	TXEN	Reserved

When this bit is 1, the data written to the THR is output on the TXD pin.
If this bit is cleared to 0 while a character is being sent, the transmission of that character is completed, but no further characters are sent until this bit is set again.
In other words, a 0 in this bit blocks the transfer of characters.

Steps to configure UART:

1. Power up the UART peripheral
2. Configure the peripheral clock (PCLK)
3. Select the UART function for GPIO pins
4. Configure the baud rate
5. Set data format and enable FIFOs
6. Transmit and receive data

Program:

```
#include <LPC17xx.h>

#define UART0_BAUDRATE 9600

void UART0_Init(void) {
   uint32_t Fdiv;

   // Enable power to UART0
   LPC_SC->PCONP |= (1 << 3);

   // Configure P0.2 as TXD0 and P0.3 as RXD0
   LPC_PINCON->PINSEL0 &= ~((3 << 4) | (3 << 6));
   LPC_PINCON->PINSEL0 |= (1 << 4) | (1 << 6);

   // Calculate baud rate divisor
   Fdiv = (25000000 / (16 * UART0_BAUDRATE));

   // Configure UART0: 8-bit data, 1 stop bit, no parity, DLAB = 1
   LPC_UART0->LCR = 0x83;
   LPC_UART0->DLM = Fdiv / 256;
   LPC_UART0->DLL = Fdiv % 256;
   LPC_UART0->LCR = 0x03;  // DLAB = 0

   // Enable FIFO
   LPC_UART0->FCR = 0x07;
}

void UART0_SendChar(char c) {
   while (!(LPC_UART0->LSR & (1 << 5)));  // Wait until THR is empty
   LPC_UART0->THR = c;
}

void UART0_SendString(const char* str) {
   while (*str) {
      UART0_SendChar(*str++);
   }
}

int main(void) {
   int i;  //  Declare variables at the top of the function

   UART0_Init();
```

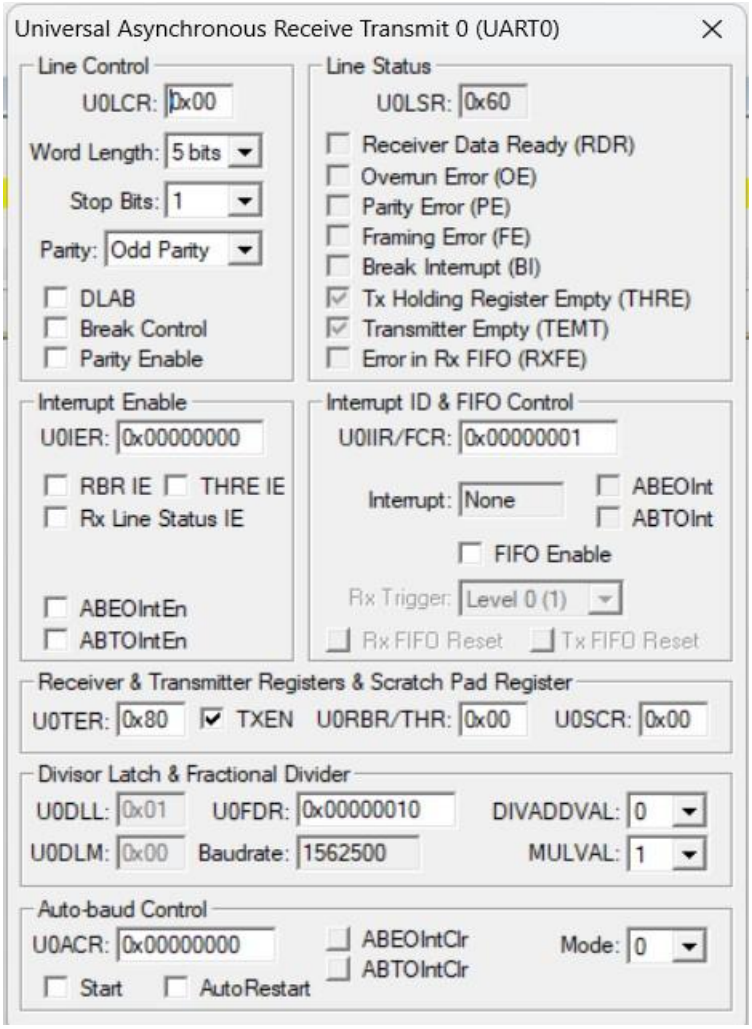
```
    while (1) {
        UART0_SendString("INTERSTELLAR\r\n");
        for (i = 0; i < 1000000; i++);  // Simple delay loop
    }
}
```

Steps for execution:

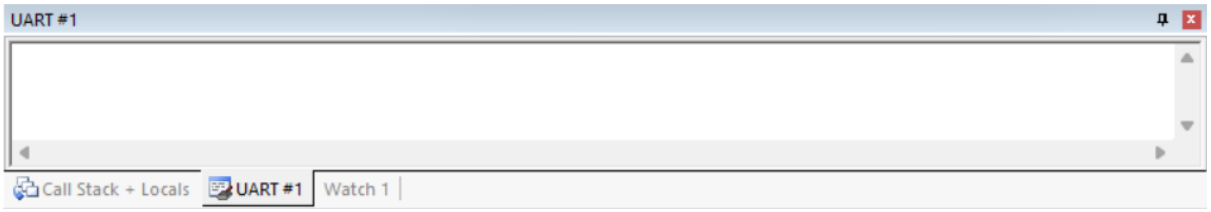
1. Steps for all embedded C programs remain the same.
2. Before Debug insert two breakpoints in the program.(Shown below, to halt execution at these points- line 19 and 45)

```
15      // Calculate baud rate divisor
16      Fdiv = (25000000 / (16 * UART0_BAUDRATE));
17
18      // Configure UART0: 8-bit data, 1 stop bit, no parity, DLAB = 1
19      LPC_UART0->LCR = 0x83;
20      LPC_UART0->DLM = Fdiv / 256;
21      LPC_UART0->DLL = Fdiv % 256;
22      LPC_UART0->LCR = 0x03;  // DLAB = 0
23
24      // Enable FIFO
25      LPC_UART0->FCR = 0x07;
26  }
27
28  void UART0_SendChar(char c) {
29      while (!(LPC_UART0->LSR & (1 << 5)));  // Wait until THR is empty
30      LPC_UART0->THR = c;
31  }
32
33  void UART0_SendString(const char* str) {
34      while (*str) {
35          UART0_SendChar(*str++);
36      }
37  }
38
39  int main(void) {
40      int i;  //  Declare variables at the top of the function
41
42      UART0_Init();
43
44      while (1) {
45          UART0_SendString("INTERSTELLAR\r\n");
46          for (i = 0; i < 1000000; i++);  // Simple delay loop
47      }
48  }
49
```

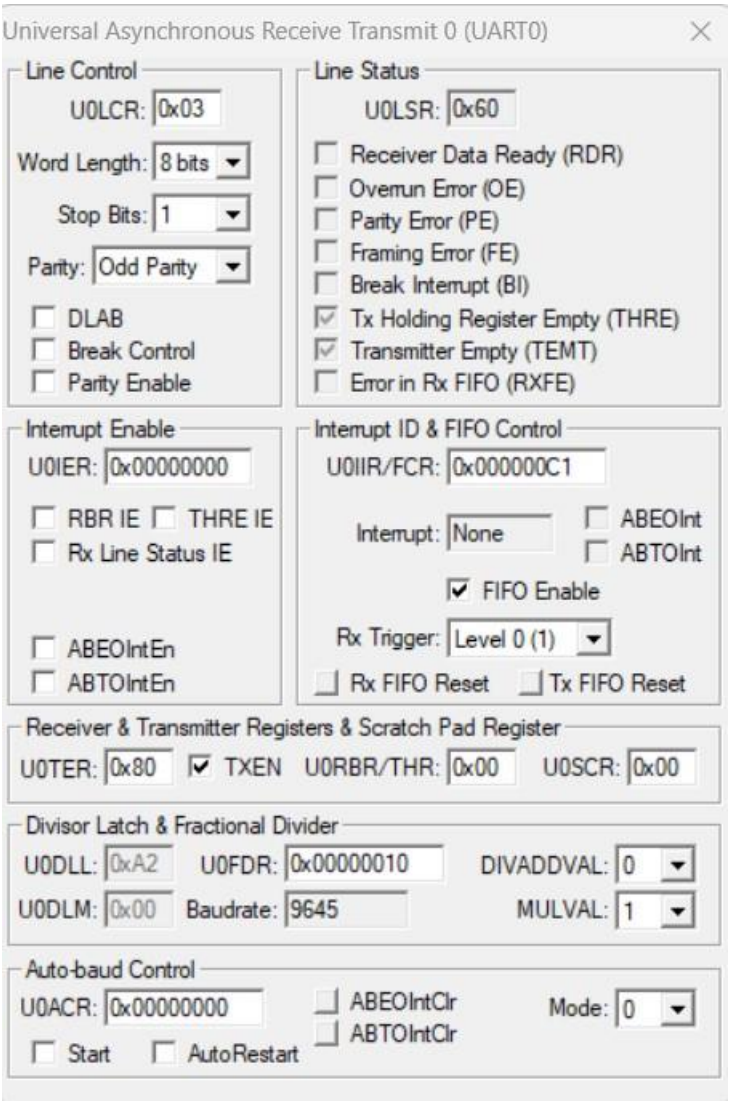
3. Goto → Peripherals → UART → UART0.

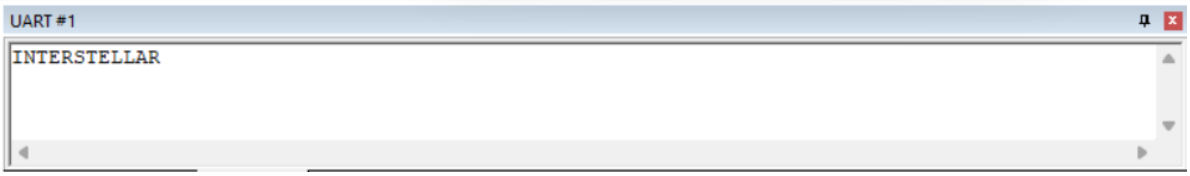


4. Goto → View → Serial windows → UART#1



5. Run the program and observe the changes in UART0 at each breakpoint.


Universal Asynchronous Receive Transmit 0 (UART0)
Line Control
U0LCR: 0x03
Word Length: 8 bits
Stop Bits: 1
Parity: Odd Parity
DLAB
Break Control
Parity Enable
Line Status
U0LSR: 0x60
Receiver Data Ready (RDR)
Overrun Error (OE)
Parity Error (PE)
Framing Error (FE)
Break Interrupt (BI)
Tx Holding Register Empty (THRE)
Transmitter Empty (TEMT)
Error in Rx FIFO (RXFE)
Interrupt Enable
U0IER: 0x00000000
RBR IE
THRE IE
Rx Line Status IE
ABEOIntEn
ABTOIntEn
Interrupt ID & FIFO Control
U0IIR/FCR: 0x000000C1
Interrupt: None
ABEOInt
ABTOInt
FIFO Enable
Rx Trigger: Level 0 (1)
Rx FIFO Reset
Tx FIFO Reset
Receiver & Transmitter Registers & Scratch Pad Register
U0TER: 0x80
TXEN
U0RBR/THR: 0x00
U0SCR: 0x00
Divisor Latch & Fractional Divider
U0DLL: 0xA2
U0FDR: 0x00000010
DIVADDVAL: 0
U0DLM: 0x00
Baudrate: 9645
MULVAL: 1
Auto-baud Control
U0ACR: 0x00000000
ABEOIntClr
ABTOIntClr
Mode: 0
Start
AutoRestart


UART #1
INTERSTELLAR

Baud rate calculation:

- Peripheral Clock(Pclk) = 25MHz(For LPC1768)
- UART Baud rate →

$$\text{BaudRate} = \frac{\text{PCLK}}{16 \times (DLL + DLM \times 256)}$$

- Fdiv = DLL+DLM*256

$$Fdiv = \frac{\text{PCLK}}{16 \times \text{BaudRate}}$$

- For Baud rate = 9600

$$Fdiv = \frac{25000000}{16 \times 9600}$$

- Fdiv = 162.76 =162(Only integer part is considered)
- These values are stored in:
  - `DLL = Fdiv % 256` (Lower 8 bits).
  - `DLM = Fdiv / 256` (Higher 8 bits).
- **DLL (Divisor Latch LSB)** → Least Significant Byte of the baud rate divisor.
- **DLM (Divisor Latch MSB)** → Most Significant Byte of the baud rate divisor.
- This calculation is indicated in program line:

Fdiv = (25000000 / (16 * UART0_BAUDRATE));

Experiment 9

AIM: **Write an embedded C program to demonstrate the working of Inter Integrated Circuit (I^2C) using LPC1768 and verify the same in Keil simulator.**

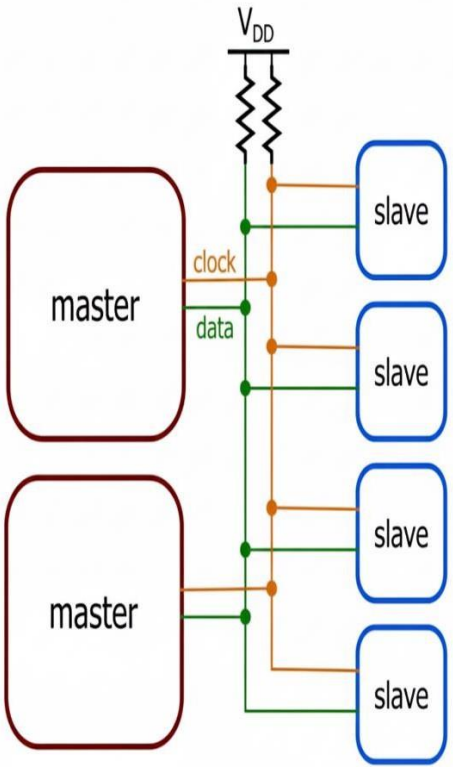
Theory: I2C (Inter Integrated Circuit) is a serial bus interface connection protocol. It is also called as TWI (two wire interface) since it uses only two wires for communication. Those two wires are SDA (serial data) and SCL (serial clock). I2C is acknowledgement-based communication protocol i.e. transmitter waits for acknowledgement from receiver after transmitting data to know whether data is received by receiver successfully.

I2C works in two modes namely,

- Master mode
- Slave mode

SDA (serial data) wire is used for data exchange in between master and slave device. SCL (serial clock) is used for synchronous clock in between master and slave device.

Master device initiates communication with slave device. Master device requires slave device address to initiate conversation with slave device. Slave device responds to master device when it is addressed by master device.



LPC1768 I2C Registers:

Register	Address (I2C0 / I2C1 / I2C2)	Width	Description	Key Bits / Values
I2CONSET (Control Set)	0x4001C000 / 0x4005C000 / 0x400A0000	8-bit	Controls operation of I²C	**AA (2):** Assert ACK **SI (3):** I²C interrupt **STO (4):** STOP flag **STA (5):** START flag **I2EN (6):** I²C Enable
I2STAT (Status)	0x4001C004 / 0x4005C004 / 0x400A0004	8-bit	Read-only status of I²C bus	**0x08:** START sent **0x10:** Repeated START **0x18:** SLA+W ACK **0x20:** SLA+W NACK **0x28:** Data ACK **0x30:** Data NACK **0x38:** Arbitration lost
I2DAT (Data)	0x4001C008 / 0x4005C008 / 0x400A0008	8-bit	Data register for transmit/receive	Holds 8-bit data value
I2ADR0 – I2ADR3 (Address Registers)	e.g. 0x4001C00C (I2C0)	8-bit each	Slave addresses for the controller	**Bits 7:1:** Slave Address **Bit 0:** GC (General Call enable)
I2SCLH (SCL High Time)	0x4001C010 / 0x4005C010 / 0x400A0010	16-bit	Defines HIGH period of SCL clock	Used with **I2SCLL** for clock rate
I2SCLL (SCL Low Time)	0x4001C014 / 0x4005C014 / 0x400A0014	16-bit	Defines LOW period of SCL clock	Together with **I2SCLH** sets SCL frequency
I2CONCLR (Control Clear)	0x4001C018 / 0x4005C018 / 0x400A0018	8-bit	Clears control bits	**AAC (2):** Clear AA **SIC (3):** Clear SI **STAC (5):** Clear START **I2ENC (6):** Disable I²C

CLOCK FREQUENCY CALCULATIONS:

- SCL frequency = PCLK / (I2SCLH + I2SCLL)
- I2CSCLH: SCLH High time
- I2SCLL : SCLL Low time
- Serial clock frequency = SCL = 100KHz
- Peripheral clock frequency = PCLK = 25MHz
- Substituting in the equation gives → I2SCLH+I2SCLL=250
- Therefore, we take I2SCLH=I2SCLL=125 → Indicated in below lines

```
LPC_I2C1->I2SCLH = 125;  // Configure clock for 100 kHz (depends on PCLK)
LPC_I2C1->I2SCLL = 125.
```

PROGRAM:

```
#include <LPC17xx.h>

#define I2C_CLK         100000    // 100 kHz standard I2C clock
#define I2C_SLAVE_ADDR  0x50      // Example slave address

void I2C_Init(void) {
   LPC_PINCON->PINSEL1 |= (1 << 22) | (1 << 24);  // P0.27 = SCL, P0.28 = SDA//(used for hardware
settings only)

   LPC_I2C1->I2SCLH = 125;   // Configure clock for 100 kHz (depends on PCLK)
   LPC_I2C1->I2SCLL = 125;

   LPC_I2C1->I2CONSET = (1 << 6);   // Enable I2C interface
}

void I2C_Start(void) {
   LPC_I2C1->I2CONSET = (1 << 5);   // Set START flag
   LPC_I2C1->I2CONCLR = (1 << 3);   // Clear SI flag

   while (!(LPC_I2C1->I2CONSET & (1 << 3)));  // Wait until SI is set

}

void I2C_Stop(void) {
   LPC_I2C1->I2CONSET = (1 << 4);   // Set STOP flag
   LPC_I2C1->I2CONCLR = (1 << 3);   // Clear SI flag
}

void I2C_WriteByte(uint8_t data) {
   LPC_I2C1->I2DAT = data;
   LPC_I2C1->I2CONCLR = (1 << 3);  // Clear SI flag

   while (!(LPC_I2C1->I2CONSET & (1 << 3)));  // Wait until SI is set
}

int main(void) {
   volatile int i;   // Declare at the top of the function

   SystemInit();
   I2C_Init();

   while (1) {
      I2C_Start();

      I2C_WriteByte((I2C_SLAVE_ADDR << 1) | 0);  // Slave address + Write (0)
      I2C_WriteByte(0xCD);   // Example data byte

      I2C_Stop();

      // Simple delay loop
      for (i = 0; i < 1000000; i++);
   }

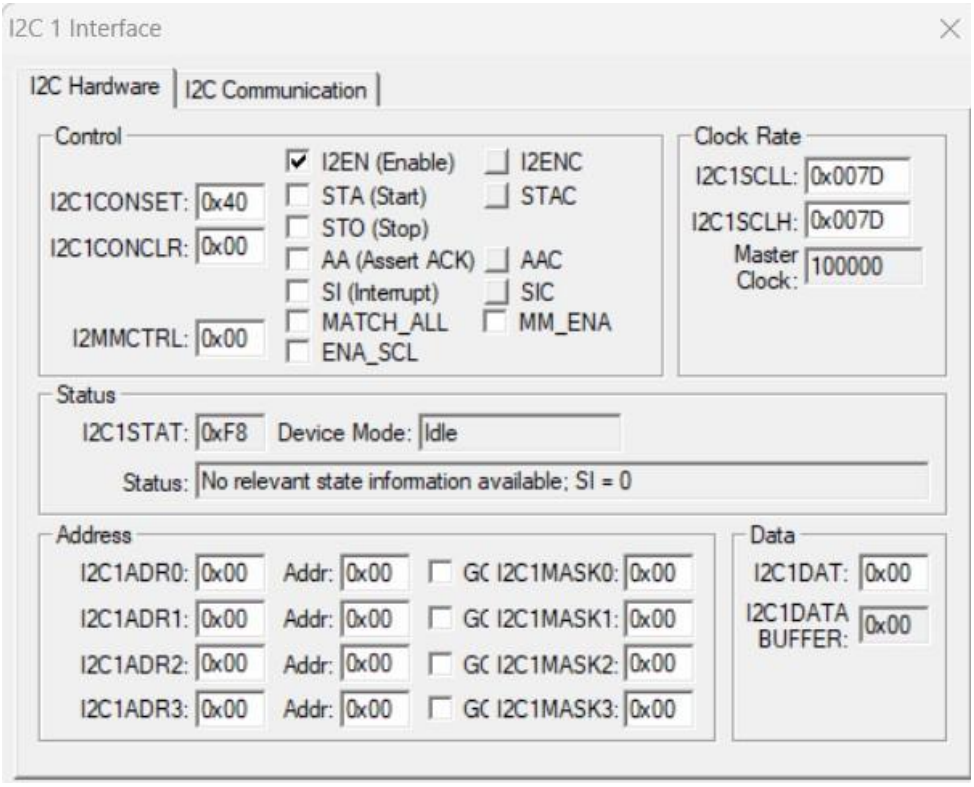
   return 0;
}
```

STEPS FOR EXECUTION:

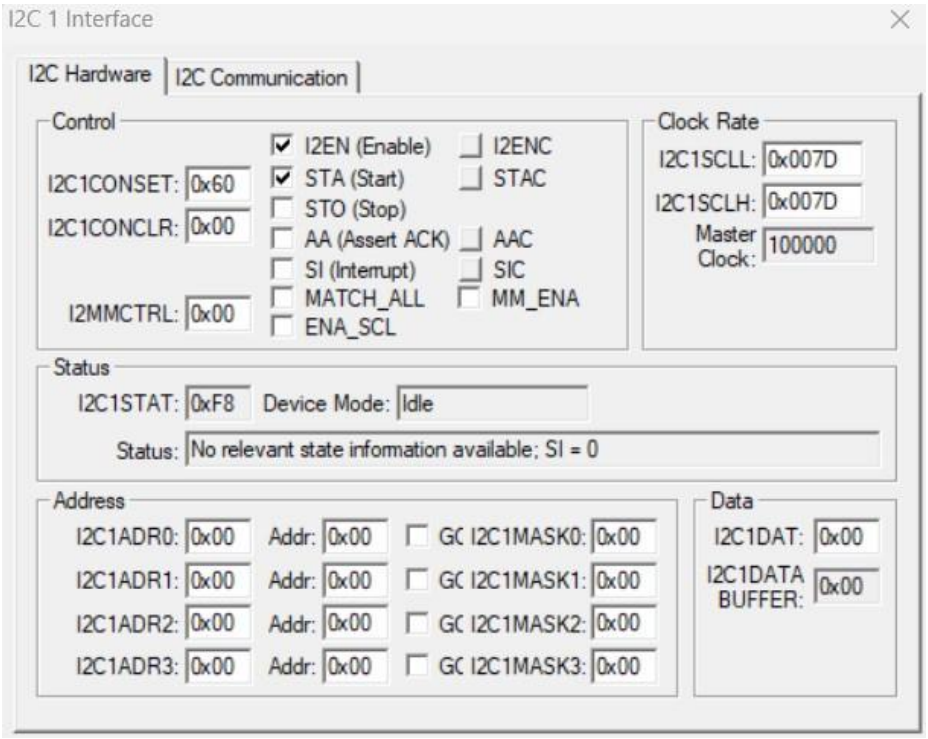
1. All embedded C program execution steps remain the same.
2. To observe the changes, we halt the execution by adding breakpoints in following lines shown below: (line 16 and 44)

```
program11a.c | startup_LPC17xx.s | system_LPC17xx.c
12      LPC_I2C1->I2CONSET = (1 << 6);   // Enable I2C interface
13  }
14
15  void I2C_Start(void) {
16      LPC_I2C1->I2CONSET = (1 << 5);   // Set START flag
17      LPC_I2C1->I2CONCLR = (1 << 3);   // Clear SI flag
18
19      while (!(LPC_I2C1->I2CONSET & (1 << 3)));  // Wait until SI is set
20
21  }
22
23  void I2C_Stop(void) {
24      LPC_I2C1->I2CONSET = (1 << 4);   // Set STOP flag
25      LPC_I2C1->I2CONCLR = (1 << 3);   // Clear SI flag
26  }
27
28  void I2C_WriteByte(uint8_t data) {
29      LPC_I2C1->I2DAT = data;
30      LPC_I2C1->I2CONCLR = (1 << 3);  // Clear SI flag
31
32      while (!(LPC_I2C1->I2CONSET & (1 << 3)));  // Wait until SI is set
33  }
34
35  int main(void) {
36      volatile int i;   // Declare at the top of the function
37
38      SystemInit();
39      I2C_Init();
40
41      while (1) {
42          I2C_Start();
43
44          I2C_WriteByte((I2C_SLAVE_ADDR << 1) | 0);  // Slave address + Write (0)
45          I2C_WriteByte(0xCD);   // Example data byte
46
```

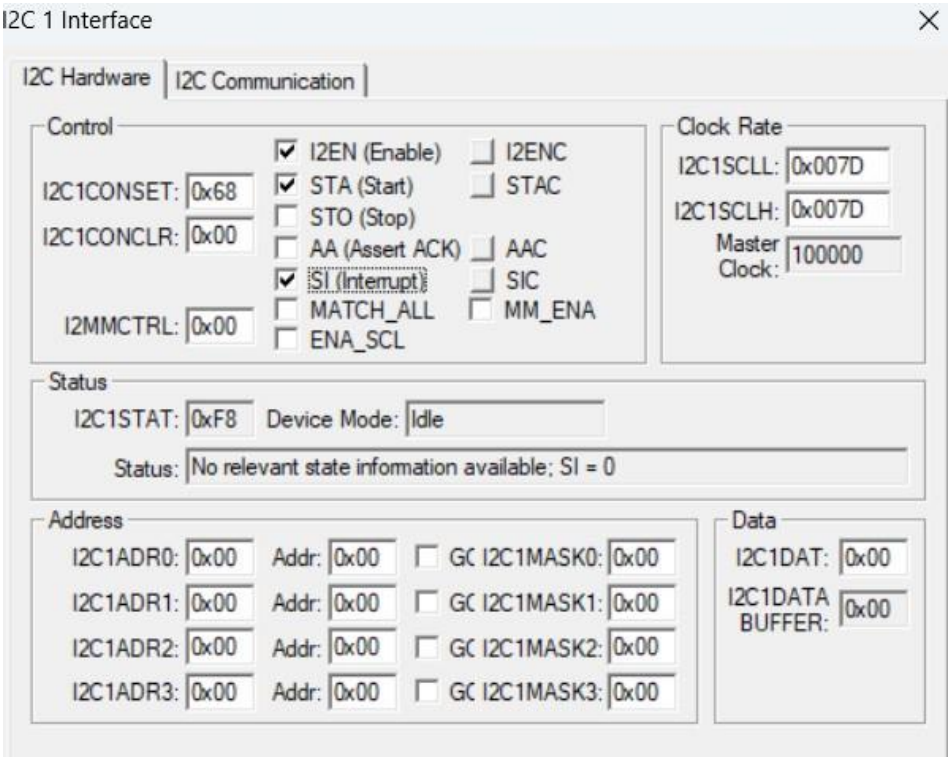
3. In Debug window goto→ Peripherals → I2C Interface and run the program. I2EN is enabled.



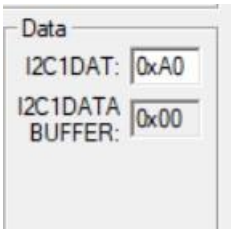
4. Run the program step by step. Next start bit is enabled.



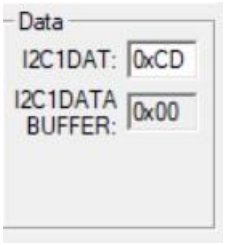
5. Run step by step and enable the SI Flag in control register.



6. Run the program step by step – Slave address is written in data register.
   Indicated in line
   ➔ I2C_WriteByte((I2C_SLAVE_ADDR << 1) | 0); // Slave address + Write (0)



7. Run the program step by step – Enable the Serial Interrupt flag again for data transfer.



Note: The program is written for simulator ouput hence, ACK, NACK, STOP are not indicated in status register.

Experiment 10

AIM: Write a C program for the ARM Cortex-M3 (LPC1768) microcontroller to demonstrate the working of special interrupt mask registers — **PRIMASK, BASEPRI, and FAULTMASK.**

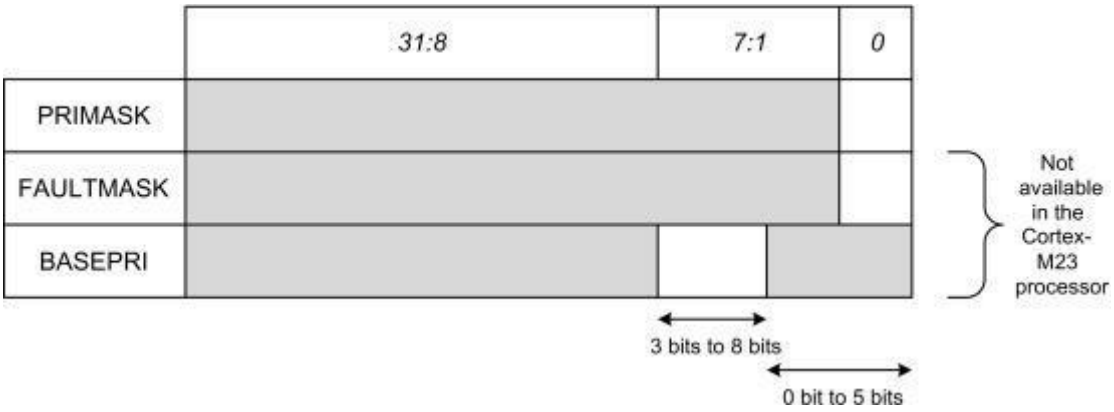
Theory:

There are three types of exception and interrupt mask registers available in the Armv8-M architecture:

- 1-bit exception mask register, PRIMASK
- 1-bit fault mask register, FAULTMASK
- 8-bit base priority mask register, BASEPRI

Each exception (including interrupts) has a priority level. A smaller number is a higher priority and a larger number is a lower priority. These special-purpose mask registers are used to mask exceptions based on priority levels. These mask registers play a vital role in priority boosting methods for exception handling within the processor. An example of the use of these registers would be to disable exceptions when they might affect timing-critical tasks.

Out of reset, bit fields in these registers are set to zero. This means that masking or disabling of exceptions and interrupts is not active out of reset.



1. PRIMASK: The PRIMASK register is a 1-bit interrupt mask register. When set, it blocks all exceptions (including interrupts) apart from Non-Maskable Interrupts (NMI) and HardFault exceptions. This effectively means that the current exception priority level is 0. Note that 0 is the highest level for a programmable exception or interrupt.
2. FAULTMASK: Similar to the PRIMASK register, the FAULTMASK register also blocks all exceptions including HardFault exception. This effectively means that the current exception priority level is -1. Typically, NMI can preempt the program when the exception priority level is -1.
3. BASEPRI: To allow flexible interrupt masking, the Armv8-M architecture provides the BASEPRI register which masks exceptions and interrupts based on priority level. The

width of the BASEPRI register depends on how many priority levels are implemented. When BASEPRI is set to 0, it means that it is disabled. When BASEPRI is set to a nonzero value, it blocks exceptions that have the same or lower priority level while allowing exceptions with a higher priority level.

System Tick Timer Calculations:

- SysTick_Config() is a CMSIS helper function that:
  - Loads the SysTick reload value = SystemCoreClock / 100.
  - Enables SysTick interrupt and counter.
- If SystemCoreClock = 100 MHz, then 100 MHz / 100 = 1,000,000 cycles → **10 ms period**.

So the **LED toggles every 10 ms**, i.e. 50 times per second.

Indicated in line → SysTick_Config(SystemCoreClock / 100);

Program:

```
#include <LPC17xx.h>
#include <core_cm3.h>   // CMSIS core functions like __get_PRIMASK, __set_PRIMASK

#define LED_PIN (1 << 22)   // P0.22 LED

/* --------- LED Control --------- */
void LED_Init(void) {
   LPC_GPIO0->FIODIR |= LED_PIN;   // Set P0.22 as output
}
void LED_Toggle(void) {
   LPC_GPIO0->FIOPIN ^= LED_PIN;
}

/* ---------- Delay Function ---------- */
void delay(void) {
   volatile int i;  // declare outside the loop
   for (i = 0; i < 500000; i++) {
      // simple software delay
   }
}

/* ---------- SysTick Interrupt ---------- */
void SysTick_Handler(void) {
   LED_Toggle();  // Toggle LED every tick
}
```

```
/* ------------ Main Program ------------- */
int main(void) {
   SystemInit();  // CMSIS system init
   LED_Init();

   // Setup SysTick: interrupt every 10ms
   SysTick_Config(SystemCoreClock / 100);

   while (1) {
      /* --- Demonstrate PRIMASK ---- */
      __set_PRIMASK(1);      // disable all maskable IRQs
      delay();               // LED will stop toggling
      __set_PRIMASK(0);      // re-enable
      delay();               // LED will toggle again

      /* --- Demonstrate BASEPRI ---- */
      __set_BASEPRI(0x40);  // mask interrupts >= priority 0x40
      delay();               // SysTick may be masked (LED stops)
      __set_BASEPRI(0x00);  // clear priority mask
      delay();

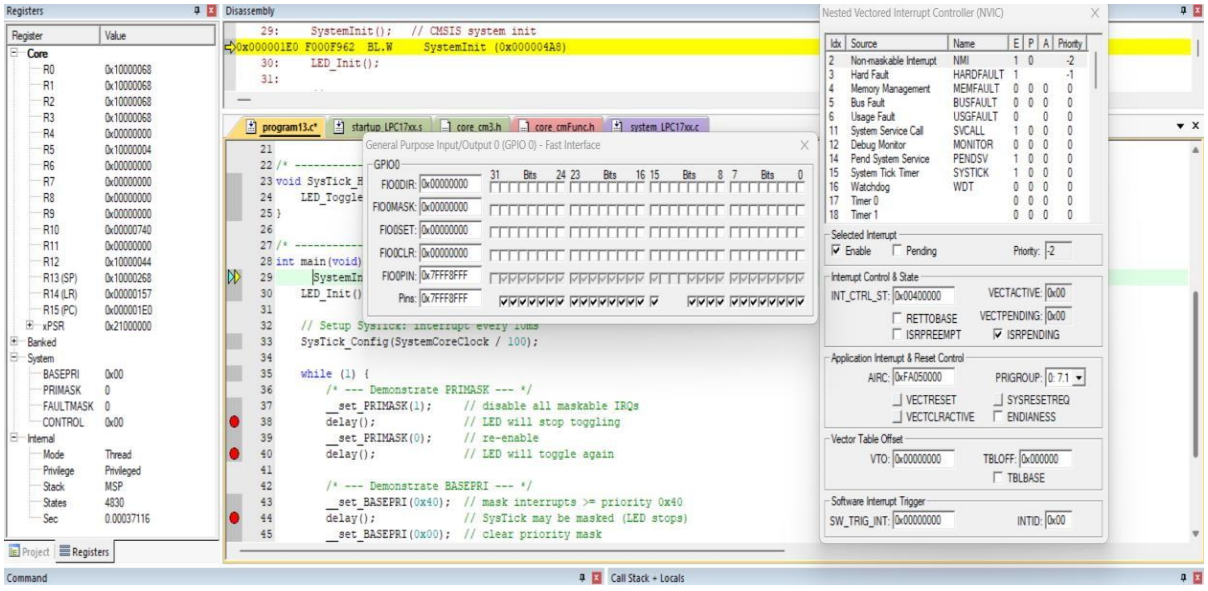
      /* --- Demonstrate FAULTMASK --- */
      __set_FAULTMASK(1);   // mask all interrupts (except NMI)
      delay();               // LED will stop toggling
      __set_FAULTMASK(0);   // restore normal operation
      delay();
   }
}
```

STEPS FOR EXECUTION:

1. All embedded C program execution steps remain the same.
2. To observe the changes, we halt the execution by adding breakpoints in following lines shown below:

```
21
22 /* ------------ SysTick Interrupt ------------ */
23 void SysTick_Handler(void) {
24     LED_Toggle();  // Toggle LED every tick
25 }
26
27 /* ------------ Main Program ------------ */
28 int main(void) {
29     SystemInit();   // CMSIS system init
30     LED_Init();
31
32     // Setup SysTick: interrupt every 10ms
33     SysTick_Config(SystemCoreClock / 100);
34
35     while (1) {
36         /* --- Demonstrate PRIMASK --- */
37         __set_PRIMASK(1);     // disable all maskable IRQs
38         delay();              // LED will stop toggling
39         __set_PRIMASK(0);     // re-enable
40         delay();              // LED will toggle again
41
42         /* --- Demonstrate BASEPRI --- */
43         __set_BASEPRI(0x40);  // mask interrupts >= priority 0x40
44         delay();              // SysTick may be masked (LED stops)
45         __set_BASEPRI(0x00);  // clear priority mask
46         delay();
47
48         /* --- Demonstrate FAULTMASK --- */
49         __set_FAULTMASK(1);   // mask all interrupts (except NMI)
50         delay();              // LED will stop toggling
51         __set_FAULTMASK(0);   // restore normal operation
52         delay();
53     }
54 }
55
```

3. In Debug window goto→ GPIO Fast interface, Peripherals → NVIC, Register window → Expand System registers. (To view PRIMASK, FAULTMASK, BASEPRI)



4. Run the program(F5) and observe the changes at each breakpoint:

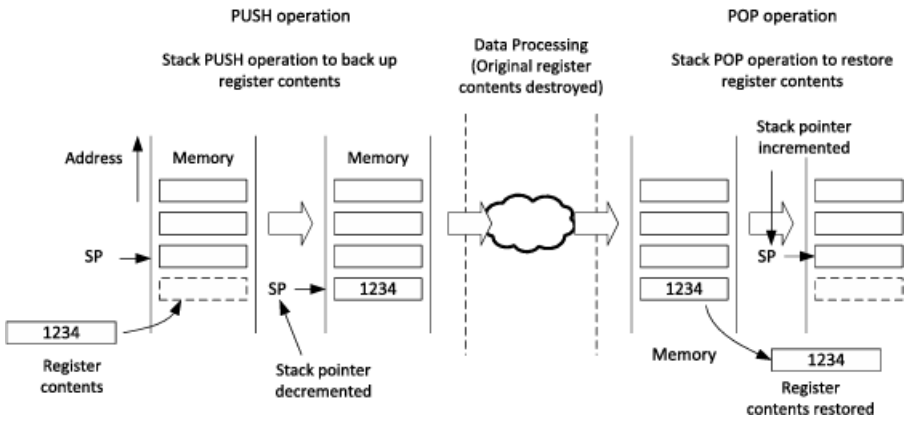
Section	Register Change	Effect	Observation
__set_PRIMASK(1)	PRIMASK = 1	Disables all IRQs	SysTick stops (P0.22 stops toggling)
__set_PRIMASK(0)	PRIMASK = 0	Re-enables IRQs	SysTick resumes, LED toggles
__set_BASEPRI(0x40)	BASEPRI = 0x40	Masks interrupts >= priority 0x40	LED may stop if SysTick priority matches
__set_BASEPRI(0x00)	BASEPRI = 0	Clears mask	LED resumes
__set_FAULTMASK(1)	FAULTMASK = 1	Blocks all except NMI	LED stops
__set_FAULTMASK(0)	FAULTMASK = 0	Normal operation	LED resumes

Experiment 11

AIM: **Write an embedded C program to demonstrate the working of two-stack model in ARM- LPC1768.**

Theory: The processor uses a full descending stack. This means that the stack pointer holds the address of the last stacked item in memory. When the processor pushes a new item onto the stack, it decrements the stack pointer and then writes the item to the new memory location. The processor implements two stacks, the main stack and the process stack, with a pointer for each held in independent registers, see Stack Pointer.

The most common uses for PUSH and POP instructions are to save contents of register banks when a function/subroutine call is made. At the beginning of the function call, the contents of some of the registers can be saved to the stack using the PUSH instruction, and then restored to their original values at the end of the function using the POP instruction**.**

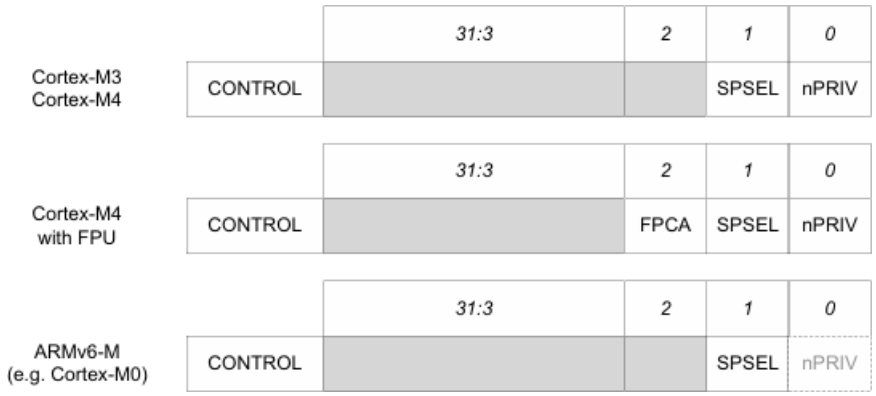


Physically there are two stack pointers in the Cortex-M processors. They are the:

• **Main Stack Pointer (MSP)** : This is the default stack pointer used after reset, and is used for all exception handlers.

• **Process Stack Pointer (PSP)** : This is an alternate stack point that can only be used in Thread mode. It is usually used for application tasks in embedded systems running an embedded OS**.**

In Thread mode, the CONTROL register controls whether the processor uses the main stack or the process stack, see CONTROL register.



		31:3	2	1	0
Cortex-M3 Cortex-M4	CONTROL			SPSEL	nPRIV
Cortex-M4 with FPU	CONTROL		FPCA	SPSEL	nPRIV
ARMv6-M (e.g. Cortex-M0)	CONTROL			SPSEL	nPRIV

Table 4.3 Bit Fields in CONTROL Register

Bit	Function
nPRIV (bit 0)	Defines the privileged level in Thread mode: When this bit is 0 (default), it is privileged level when in Thread mode. When this bit is 1, it is unprivileged when in Thread mode. In Handler mode, the processor is always in privileged access level.
SPSEL (bit 1)	Defines the Stack Pointer selection: When this bit is 0 (default), Thread mode uses Main Stack Pointer (MSP). When this bit is 1, Thread mode uses Process Stack Pointer (PSP). In Handler mode, this bit is always 0 and write to this bit is ignored.
FPCA (bit 2)	Floating Point Context Active – This bit is only available in Cortex-M4 with floating point unit implemented. The exception handling mechanism uses this bit to determine if registers in the floating point unit need to be saved when an exception has occurred. When this bit is 0 (default), the floating point unit has not been used in the current context and therefore there is no need to save floating point registers. When this bit is 1, the current context has used floating point instructions and therefore need to save floating point registers. The FPCA bit is set automatically when a floating point instruction is executed. This bit is clear by hardware on exception entry. There are several options for handling saving of floating point registers. This will be covered in Chapter 13.

PROGRAM

```
#include <LPC17xx.h>
#include <core_cm3.h>   // CMSIS core access functions

uint32_t processStack[128];         // Process Stack (512 bytes = 128 words)
uint32_t *psp_top;                  // Pointer to top of process stack

uint32_t msp_before, psp_before, control_before;
uint32_t control_after, current_psp, current_msp;

void delay(void) {
   volatile int i;
   for (i = 0; i < 200000; i++);   // Simple software delay
```

```
}

int main(void) {


   LPC_GPIO0->FIODIR |= (1 << 22);     // Set P0.22 as output


   msp_before = __get_MSP();           // Read Main Stack Pointer
   psp_before = __get_PSP();           // Read Process Stack Pointer
   control_before = __get_CONTROL();  // Read CONTROL register (bit1=stack select)


   psp_top = processStack + 128;       // Top of process stack (stack grows down)
   __set_PSP((uint32_t)psp_top);       // Load PSP with new address

   __set_CONTROL(0x02);                // Bit1=1 use PSP, Bit0=0  privileged mode
   __ISB();                            // Instruction Synchronization Barrier

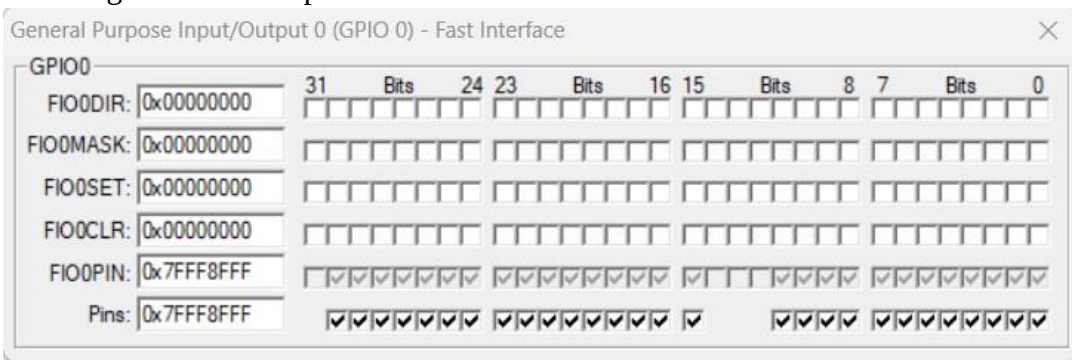

   control_after = __get_CONTROL();   // CONTROL after switching
   current_psp   = __get_PSP();        // PSP now active
   current_msp   = __get_MSP();        // MSP unchanged
   while (1) {
      LPC_GPIO0->FIOPIN ^= (1 << 22); // Toggle LED
      delay();
   }
}
```

STEPS TO EXECUTE:

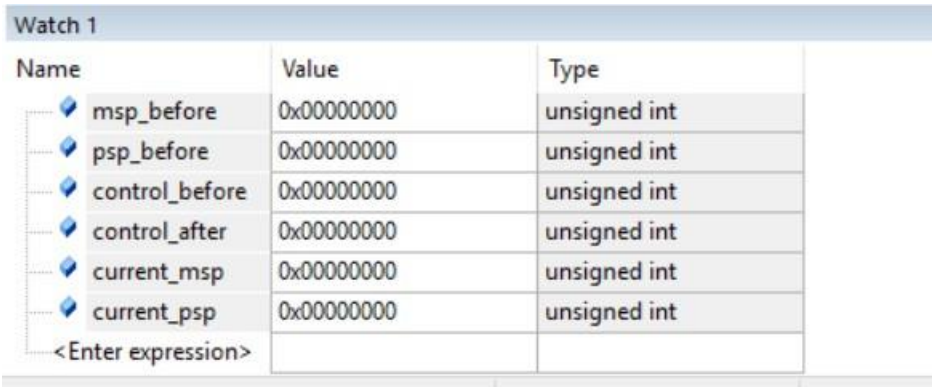
1. All C program steps remain the same.
2. Add breakpoints in the program to observe changes

```
 7 uint32_t *psp_top;                  // Pointer to top of process stack
 8
 9 uint32_t msp_before, psp_before, control_before;
10 uint32_t control_after, current_psp, current_msp;
11
12 /* ------------------ Delay Function ------------------ */
13 void delay(void) {
14     volatile int i;
15     for (i = 0; i < 200000; i++);   // Simple software delay
16 }
17
18 /* ------------------ Main Function ------------------ */
19 int main(void) {
20
21     /* Step 0: Configure LED on P0.22 */
22     LPC_GPIO0->FIODIR |= (1 << 22);     // Set P0.22 as output
23
24     /* Step 1: Read current stack pointers and control register */
25     msp_before = __get_MSP();           // Read Main Stack Pointer
26     psp_before = __get_PSP();           // Read Process Stack Pointer
27     control_before = __get_CONTROL();  // Read CONTROL register (bit1=stack select)
28
29     /* Step 2: Initialize PSP (Process Stack Pointer) */
30     psp_top = processStack + 128;       // Top of process stack (stack grows down)
31     __set_PSP((uint32_t)psp_top);       // Load PSP with new address
32
33     /* Step 3: Switch Thread Mode to use PSP instead of MSP */
34     __set_CONTROL(0x02);                // Bit1=1 ? use PSP, Bit0=0 ? privileged mode
35     __ISB();                            // Instruction Synchronization Barrier
36
37     /* Step 4: Read back to confirm change */
38     control_after = __get_CONTROL();   // CONTROL after switching
39     current_psp   = __get_PSP();        // PSP now active
40     current_msp   = __get_MSP();        // MSP unchanged
41
```

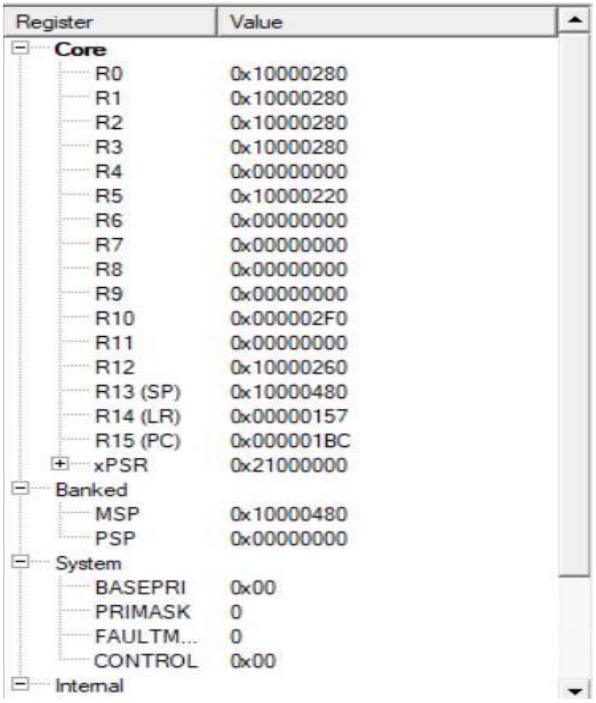
3. Run the program at once till breakpoint and run step by step to observe the values
4. In Debug window → Open GPIO Fast interface → GPIO0



5. Open Watch window → Enter the following variables →
   - msp_before
   - psp_before
   - control_before
   - control_after
   - current_psp
   - current_msp



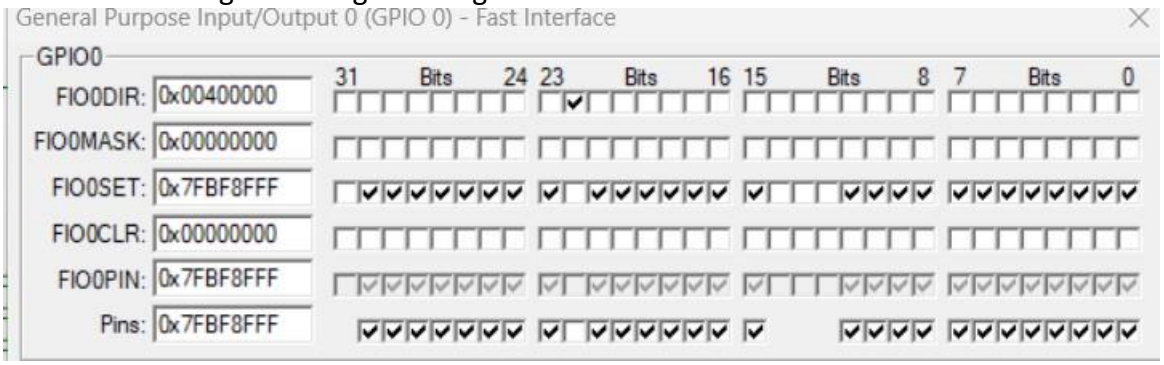
6. In Register window → Expand Banked and System Registers to observe changes



7. Summary of values is given below

Register / Parameter	Before Switching (using MSP)	After Switching (using PSP)	Explanation
MSP (Main Stack Pointer)	e.g., 0x10007FF8 *(varies)*	e.g., 0x10007FF8 *(unchanged)*	MSP is the default stack used at reset. It remains same because switching to PSP does not alter MSP.
PSP (Process Stack Pointer)	0x00000000	0x10000200 (or 0x10000220)	PSP initialized to point to the top of processStack[128]. Stack grows downward from this address.
CONTROL Register	0x00 → Bit[1]=0 → Using MSP	0x02 → Bit[1]=1 → Using PSP	Bit1 of CONTROL determines which stack is used in Thread mode:0 → MSP, 1 → PSP.
Processor Mode	Thread mode	Thread mode	The switch only changes the active stack, not the mode.
Stack Memory Used	Main stack region in SRAM (default)	User-defined stack array (processStack)	Indicates the two separate memory areas for stack operations.

8. LED Blinking indicating PSP usage



9. Variable values and Register window:

Watch 1

Name	Value	Type
msp_before	0x10000480	unsigned int
psp_before	0x00000000	unsigned int
control_before	0x00000000	unsigned int
control_after	0x00000002	unsigned int
current_msp	0x10000480	unsigned int
current_psp	0x10000220	unsigned int
<Enter expression>		

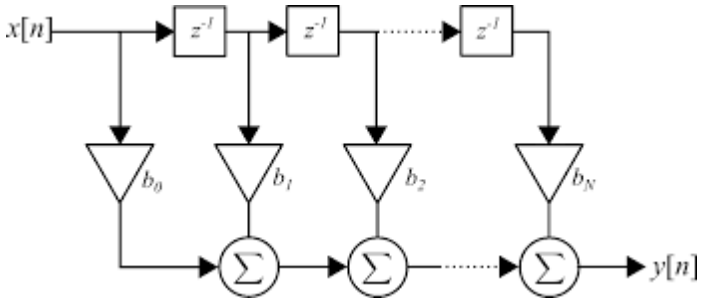
Register	Value
Core	
R0	0x00000002
R1	0x10000000
R2	0x10000280
R3	0x10000280
R4	0x00000000
R5	0x10000220
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x000002F0
R11	0x00000000
R12	0x10000260
R13 (SP)	0x10000220
R14 (LR)	0x00000157
R15 (PC)	0x00000200
xPSR	0x21000000
Banked	
MSP	0x10000480
PSP	0x10000220
System	
BASEPRI	0x00
PRIMASK	0
FAULTMASK	0
CONTROL	0x02
Internal	

10. Stack Address Calculation:
    - Initial PSP= 0x00000000
    - Stack assigned = Process stack + 512 bytes (128 words)
    - (512)d = (200)h
    - Stack address = Base address + size = 10000000+200= 0x10000200
    - To maintain **8-byte alignment** the compiler or linker added **32 bytes extra** to at the top.
    - Hence the stack address is **0x10000220**

Experiment 12

AIM: **Write an embedded C program to implement Type-II Linear FIR Filter using ARM LPC1768.**

Theory: The Finite Impulse Response (FIR) filter is a kind of digital filter whose response to an impulse input lasts for a finite time, and then it becomes zero at some number of samples taken. These filters are simple in nature and also have stability as they do not require the use of feedback. Usually, FIR filters are implemented by using coefficients that determine its behavior and can be linear phase making all the frequency components in the input signal delayed for equal periods of time.



Linear phase FIR filters are designed by choosing the coefficients h[n] to be either symmetric or anti-symmetric around the center of the impulse response.
Depending on the filter length N (number of coefficients: even or odd) and symmetry type, there are four types:

Type	N (number of taps)	Symmetry	Frequency response	Typical use
I	odd	Symmetric h[n]=h[N−1−n]	cosine response, non-zero at DC & Nyquist	LPF, HPF, BP
II	even	symmetric	zero at Nyquist	LPF, HPF (special cases)
III	odd	anti-symmetric h[n]=−h[N−1−n]	sine response, zero at DC	Hilbert transformer, differentiator
IV	even	anti-symmetric	sine response, zeros at DC & Nyquist	phase shifters

Implementation:

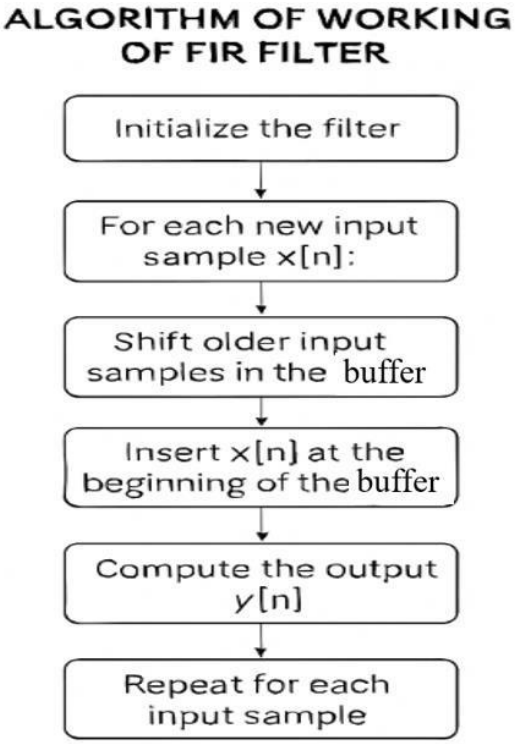
Let x[n] be the input to the filter at time n and let y[n] be the output. The output is computed using the difference equation:

$$y[n] = \sum_{k=0}^{N-1} x[n-k]h[k]$$

where h[n] are the filter coefficients. In the difference equation above, the FIR filter has N coefficients:

{h[0], h[1], ……., h[N-1]} and the output is computed using N previous input samples:

{ x[n], x[n-1], ….., x[n- (N-1)] }



Program:

```
#include <LPC17xx.h>
#define N 4 // Number of FIR filter taps
// FIR coefficients (simple moving average)
float h[N] = {0.25, 0.25, 0.25, 0.25};
// FIR input buffer
float x[N] = {0};
volatile float output=0;
volatile int i;
// Input samples (can replace with ADC input)
float input_samples[] = {100, 120, 80, 60, 200, 100, 150, 90, 50, 0};
```

```
int sample_index = 0;
int num_samples = sizeof(input_samples) / sizeof(float);
// FIR filter function
float fir_filter (float new_sample)
{
float result = 0;
int i;
for (i= N-1; i > 0; i--)
{
x[i] = x[i - 1];
}
x[0] = new_sample;
for (i=0; i < N; i++)
{
result += h[i] * x[i];
}
return result;
}
int main(void)
{
float input;
SystemInit();
SystemCoreClockUpdate();
// Configure P0.24 and P0.25 as GPIO output (relay control)
LPC_PINCON->PINSEL1 &= ~(0xF << 16); // Clear function bits
LPC_GPIO0->FIODIR |= (1<<24) | (1<<25); // Set as output
while (1) {
// Simulate sample input
if (sample_index <num_samples)

input = input_samples [sample_index++];
else
input = 0;
// Apply FIR filter
output=fir_filter (input);

// Relay ON if output > 100
if (output> 100.0) {
LPC_GPIO0->FIOSET = (1<< 24) | (1 << 25);
}
else
        {
                LPC_GPIO0->FIOCLR = (1 << 24) | (1 << 25);
}

// Short delay (for Logic Analyzer waveform spacing)
```

```
for (i=0; i < 500000; i++);
}
}
```

Filter output calculations:

Parameter	**Value**
Filter type	4-tap FIR (simple moving average)
Coefficients (h[i])	{0.25, 0.25, 0.25, 0.25}
Input samples (x[n])	{100, 120, 80, 60, 200, 100, 150, 90, 50, 0}

The filter equation is:

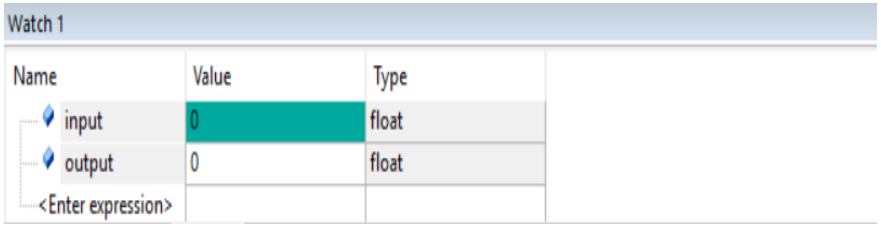
$$\boldsymbol{y[n] = 0.25(x[n] + x[n-1] + x[n-2] + x[n-3])}$$

Assume previous values are **0** before the first sample.

Sample No.	**Input (x[n])**	**x[n-1]**	**x[n-2]**	**x[n-3]**	**Output (y[n]) = 0.25×(sum)**	**Relay Status (>100 → ON)**
1	100	0	0	0	(100 + 0 + 0 + 0) × 0.25 = **25.0**	OFF
2	120	100	0	0	(120 + 100 + 0 + 0) × 0.25 = **55.0**	OFF
3	80	120	100	0	(80 + 120 + 100 + 0) × 0.25 = **75.0**	OFF
4	60	80	120	100	(60 + 80 + 120 + 100) × 0.25 = **90.0**	OFF
5	200	60	80	120	(200 + 60 + 80 + 120) × 0.25 = **115.0**	**ON**
6	100	200	60	80	(100 + 200 + 60 + 80) × 0.25 = **110.0**	**ON**
7	150	100	200	60	(150 + 100 + 200 + 60) × 0.25 = **127.5**	**ON**
8	90	150	100	200	(90 + 150 + 100 + 200) × 0.25 = **135.0**	**ON**
9	50	90	150	100	(50 + 90 + 150 + 100) × 0.25 = **97.5**	OFF
10	0	50	90	150	(0 + 50 + 90 + 150) × 0.25 = **72.5**	OFF

STEPS FOR EXECUTION:

1. All embedded C program execution steps remain the same.
2. Goto → Debug → Peripherals → GPIO Fast Interface → GPIO0.
3. Open Logic Analyser → Insert signal → Output
4. Open Watch window → watch 1 → type input and output to observe the changes.



5. Run the program – F5
6. Observe the output values to verify.
7. When the output is greater than 100, Led turns ON else it is OFF.

